



ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO MÔN HỌC
LẬP TRÌNH NÂNG CAO TRONG ĐO LƯỜNG ĐIỀU KHIỂN

THIẾT KẾ VÀ CHẾ TẠO TAY CÀM ĐIỀU KHIỂN HID
ĐA NĂNG

NGUYỄN VĂN PHÚ 23021322

ĐÀO MẠNH NGỌC 23021316

PHẠM GIA VINH 23021370

LÊ VĂN ĐỨC 23021248

GIẢNG VIÊN HƯỚNG DẪN:

TS. ĐỖ TRẦN THẮNG

THS. LÊ DUY MINH

HÀ NỘI, 2025

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy Lê Duy Minh và thầy Đỗ Trần Thắng – giảng viên bộ môn Lập trình nâng cao trong đo lường điều khiển. Trong suốt quá trình thực hiện bài tập lớn này, các thầy đã luôn tận tình hướng dẫn, đưa ra những gợi ý chuyên môn quý báu và tạo điều kiện để chúng em có thể áp dụng ngôn ngữ lập trình Python vào giải quyết một bài toán thực tế về thiết bị ngoại vi HID.

Em cũng xin gửi lời cảm ơn tới ban lãnh đạo cùng các thầy cô thuộc Khoa/Trường đã tạo ra môi trường học tập và nghiên cứu hiện đại, giúp em có nền tảng kiến thức vững chắc để hoàn thiện sản phẩm tay cầm điều khiển sử dụng chip ESP32-S3 này.

Mặc dù em đã dành nhiều tâm huyết và nỗ lực để hoàn thành dự án, nhưng do kiến thức và kinh nghiệm còn hạn chế, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ thầy để sản phẩm và kiến thức của em ngày càng hoàn thiện hơn.

TÓM TẮT

Báo cáo này trình bày toàn diện quá trình nghiên cứu, thiết kế và chế tạo hệ thống Tay cầm Điều khiển HID (Human Interface Device) đa năng, được xây dựng trên nền tảng vi điều khiển ESP32-S3 N16R8 với đặc tính Native USB tốc độ cao. Hệ thống được lập trình hoàn toàn bằng ngôn ngữ Python thông qua trình thông dịch CircuitPython trên thiết bị nhúng và thư viện PyQt6 trên ứng dụng máy tính.

Về phần cứng, tay cầm tích hợp hai Joystick analog cho đầu vào định hướng 360 độ, hai Rotary Encoder EC11 vô cấp hỗ trợ cuộn chuột và điều chỉnh âm lượng, cụm bảy phím chức năng được quản lý thông qua IC mở rộng I/O PCF8574 trên bus I2C, màn hình OLED SSD1306 128×64 pixel hiển thị trạng thái thời gian thực, cùng một Buzzer phản hồi âm thanh tương tác. Toàn bộ hệ thống được hàn tay trực tiếp trên bo mạch Perfboard chuyên dụng.

Về phần mềm, firmware CircuitPython trên ESP32-S3 triển khai một vòng lặp sự kiện thời gian thực (Real-time Polling Event Loop), xử lý đồng thời các luồng đầu vào từ ADC, GPIO, I2C và bộ đếm xung, sau đó chuyển đổi cấu trúc dữ liệu kết quả thành các bản tin HID Composite (Keyboard + Mouse + Consumer Control) truyền qua cổng USB-OTG. Ứng dụng máy tính PyQt6 cung cấp giao diện đồ họa trực quan cho phép người dùng ánh xạ lại (remap) tất cả các nút vật lý tới bất kỳ lệnh HID nào mà không cần can thiệp vào mã nguồn nhúng, thông qua tệp cấu hình JSON được đồng bộ hóa trực tiếp lên bộ nhớ flash của thiết bị.

Các thử nghiệm thực tế cho thấy hệ thống đạt độ trễ phản hồi đáp ứng được các mục đích sử dụng cơ bản, hoạt động ổn định liên tục và được nhận diện hoàn hảo như thiết bị HID chuẩn trên các hệ điều hành Windows, macOS và Linux mà không yêu cầu cài đặt driver bổ sung. Sản phẩm có tiềm năng ứng dụng cao trong điều khiển phần mềm đo lường, gaming, trình chiếu chuyên nghiệp và các hệ thống HMI công nghiệp nhẹ.

MỤC LỤC

LỜI CẢM ƠN	I
TÓM TẮT	1
CHƯƠNG 1 : GIỚI THIỆU ĐỀ TÀI	7
1.1. Lý do chọn đề tài	7
1.2. Mục tiêu đề tài	8
1.3. Đối tượng và phạm vi nghiên cứu	8
1.4. Phương pháp Nghiên cứu	9
1.5. Ý nghĩa khoa học và thực tiễn	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	10
2.1. Chuẩn giao tiếp USB HID	10
2.1.1. Kiến trúc tổng quan của USB HID	10
2.1.2. USB Composite Device – Thiết bị HID đa chức năng	10
2.2. Vi điều khiển ESP32-S3 N16R8	11
2.3. Giao thức I2C và các IC ngoại vi	12
2.3.1. Phân tích giao thức I2C trong sản phẩm	12
2.3.2. IC mở rộng I/O PCF8574	13
2.3.3. Màn hình OLED SSD1306	14
2.4. Rotary Encoder EC11 và kỹ thuật đọc xung Quadrature	14
2.5. CircuitPython và hệ sinh thái Adafruit HID	16
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG VÀ CHẾ TẠO PHẦN CỨNG	17
3.1. Kiến trúc tổng thể của hệ thống	17
3.2. Lựa chọn linh kiện và thông số kỹ thuật	18
3.2.1. Vi điều khiển trung tâm: ESP32-S3 (N16R8)	18
3.2.2. IC mở rộng ngoại vi: PCF8574	19

3.2.3. Khối điều khiển Analog: Joystick KY-023	19
3.2.4. Khối tinh chỉnh: Rotary Encoder EC11	20
3.2.5. Khối hiển thị và phản hồi: OLED SSD1306 & Buzzer	20
3.3. Nguyên lý hoạt động của sản phẩm	21
3.3.1. Sơ đồ luồng dữ liệu (Data Flow)	21
3.3.2. Nguyên lý xử lý các khối chức năng	22
A. Khối nút nhấn và Giao thức I2C (PCF8574A)	22
B. Khối Joystick (Analog to Digital Converter - ADC)	23
C. Khối Rotary Encoder (Xung điện từ)	23
D. Hiện tượng dội phím cơ học (Bouncing)	23
3.4. Thiết kế mạch điện và giải pháp kỹ thuật	24
3.4.1. Giải pháp ổn định giao tiếp I2C đa thiết bị	25
3.4.2. Quản lý I/O và tối ưu hóa tốc độ phản hồi (Active Low)	25
3.4.3. Hệ thống quản lý nguồn điện và tự ổn định tín hiệu	26
3.5. Quy trình chế tạo và bố trí linh kiện	26
3.5.1. Quy trình gia công và lắp ráp	26
3.5.2. Tư duy bố trí linh kiện tối ưu trải nghiệm (Ergonomic Design)	27
CHƯƠNG 4 :KIẾN TRÚC FIRMWARE, PHẦN MỀM ĐIỀU KHIỂN VÀ MÔ HÌNH DỮ LIỆU .	28
4.1. Phân tầng thực thi	28
4.2. Giải thuật tối ưu hóa băng thông I2C	29
4.3. Giải thuật xử lý Analog và Deadzone (Ba vùng)	30
4.4. Mô hình nạp dữ liệu động (Dynamic Key Mapping)	32
4.5. Kiến trúc phần mềm và lựa chọn Framework	32
4.5.1. Lựa chọn Framework PyQt6	32
4.5.2. Áp dụng mô hình MVC (Model-View-Controller)	33

4.6. Thiết kế giao diện (UI/UX) và Đồ họa tùy chỉnh.....	33
4.6.1. Giải pháp mô phỏng đồ họa (Digital Twin).....	33
4.6.2. Giao diện ánh xạ phím và cấu trúc danh mục.....	34
4.7. Mô hình dữ liệu và Cơ chế Persistence.....	34
4.7.1. Serialization dữ liệu sang định dạng JSON.....	34
4.7.2. Dynamic Loading Workflow (Flash-free Updating).....	34
CHƯƠNG 5: KẾT QUẢ, KIỂM NGHIỆM VÀ ĐÁNH GIÁ.....	36
5.1. Kết quả đạt được.....	36
5.1.1. Sản phẩm phần cứng hoàn thiện.....	36
5.1.2. Kết quả kiểm thử chức năng.....	36
5.2. Phân tích ưu và nhược điểm của kiến trúc.....	38
5.2.1. Ưu điểm.....	38
5.2.2. Nhược điểm và hạn chế.....	39
5.3. Kết luận và định hướng phát triển.....	39
5.3.1. Kết luận.....	39
5.3.2. Định hướng phát triển.....	39

DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU

Hình 1.1.Tay cầm HID thương mại trên thị trường	7
Hình 2.1. Cơ chế hoạt động của USB HID	10
Hình 2.3.Giao thức I2C trong tay cầm	12
Hình 2.4.IC mở rộng I/O PCF8574	13
Hình 2.5.Nguyên lý đọc nút nhấn bằng PCF8574	14
Hình 2.6. Ước lượng thời gian truyền qua I2C	14
Hình 2.7.Kỹ thuật đọc xung quadrature EC11	15
Hình 2.8. Đọc xung bằng GPIO của MCU	16
Hình 2.9.Lập trình sản phẩm bằng circuitpython	16
Hình 2.10. Cấu trúc HID report và một số lệnh phổ biến	17
Hình 3.1.Kiến trúc hệ thống của tay cầm HID	17
Hình 3.2. ESP32-S3 N16R8	18
Hình 3.3.IC mở rộng ngoại vi: PCF8574	19
Hình 3.4. Joystick KY-023 2 trục	20
Hình 3.5. Encoder EC11	20
Hình 3.6. Oled 0.96 inch và buzzer	20
Hình 3.7. Sơ đồ luồng dữ liệu của tay cầm điều khiển HID đa năng	21
Hình 3.8.Hiện tượng dội phím cơ học	23
Hình 3.9. Sơ đồ nguyên lý mạch của sản phẩm	24
Hình 4.1. Lưu đồ thuật toán chính của tay cầm	28
Hình 4.3.Thuật toán đọc nhiều nút nhấn và xử lý sự kiện	30
Hình 4.4. Thuật toán deadzone joystick	31
Hình 4.5. Giao diện đồ hoạ giao diện cấu hình tay cầm	33
Hình 4.6. Lưu đồ thuật toán phần mềm giao diện điều khiển	35

Hình 5.1.Sản phẩm hoàn chỉnh	36
Hình 5.2. Laptop nhận diện mượt mà và test bằng phần mềm phản hồi nhanh	38
Hình 5.3.Sản phẩm nâng cấp dự kiến	40
Bảng 1 : Linh kiện sử dụng , thông số và vai trò trong mạch	18
Bảng 2 : Bảng tổng hợp thông số kỹ thuật hệ thống	21
Bảng 3 : Các tiêu chí kiểm thử sản phẩm và kết quả thực nghiệm	36

CHƯƠNG 1 : GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài



Hình 1.1. Tay cầm HID thương mại trên thị trường

Trong xu thế phát triển của hệ thống đo lường và điều khiển hiện đại, giao diện người – máy (HMI – Human-Machine Interface) đóng vai trò then chốt quyết định hiệu quả vận hành. Trong khi các phòng thí nghiệm và môi trường công nghiệp nhẹ thường phải chấp nhận sử dụng bàn phím và chuột máy tính thông dụng – vốn không được thiết kế tối ưu cho các tác vụ điều khiển chuyên biệt – thì các giải pháp điều khiển chuyên dụng thương mại lại có chi phí rất cao và không linh hoạt trong việc tùy chỉnh chức năng.

Chuẩn USB HID (Human Interface Device), được IEEE và USB-IF chuẩn hóa từ năm 1996 và liên tục cập nhật, cung cấp một giao thức truyền thông thiết bị ngoại vi đã được hợp nhất vào nhân của tất cả các hệ điều hành hiện đại. Lợi thế cạnh tranh của HID nằm ở tính năng Plug-and-Play tuyệt đối: hệ điều hành tự động nhận diện và nạp driver tích hợp sẵn mà không yêu cầu bất kỳ tác động can thiệp nào từ phía người dùng. Đây là đặc tính nền tảng giúp thiết bị HID tự chế trở thành lựa chọn hoàn hảo cho các dự án điều khiển tùy biến.

Sự xuất hiện của dòng vi điều khiển ESP32-S3 từ Espressif Systems đã mở ra một cơ hội đột phá. Không giống các thế hệ tiền nhiệm (ESP8266, ESP32) vốn chỉ hỗ trợ USB thông qua chip chuyển đổi ngoại vi (ví dụ CH340G), ESP32-S3 tích hợp bộ điều khiển USB OTG

Full-Speed hoàn chỉnh trực tiếp trên die silicon, cho phép giả lập thiết bị HID với độ trễ cực thấp và loại bỏ hoàn toàn lớp firmware trung gian vốn là nguồn gốc của jitter tín hiệu. Đặc biệt, biến thể N16R8 với 16MB Flash và 8MB Octal PSRAM tốc độ cao cung cấp dung lượng bộ nhớ vượt trội so với nhu cầu, cho phép nạp toàn bộ runtime CircuitPython và các thư viện HID mà không phải đánh đổi bất kỳ chức năng nào.

Về ngôn ngữ lập trình, Python thông qua trình thông dịch CircuitPython (một nhánh của MicroPython được Adafruit phát triển và tối ưu hóa cho các ứng dụng nhúng giáo dục và prototype nhanh) cho phép triển khai logic điều khiển phức tạp với tốc độ phát triển vượt trội so với C/C++. Hệ sinh thái thư viện `adafruit_hid` cung cấp sẵn các lớp trừu tượng Keyboard, Mouse, ConsumerControl được kiểm thử kỹ lưỡng và tuân thủ hoàn toàn đặc tả USB HID 1.11.

Dự án này không đơn thuần là một bài tập kỹ thuật mà còn nhằm xây dựng nền tảng cho một nền tảng điều khiển mở rộng có thể được tái cấu hình hoàn toàn thông qua phần mềm, phục vụ các ứng dụng đa dạng từ điều khiển thiết bị đo lường, phần mềm mô phỏng kỹ thuật, đến các hệ thống thực tế ảo (VR) và tăng cường (AR).

1.2. Mục tiêu đề tài

Đề tài đặt ra ba nhóm mục tiêu kỹ thuật được phân cấp theo độ ưu tiên:

- **Mục tiêu cốt lõi (Core Objectives):** Thiết kế và chế tạo thành công phần cứng tay cầm HID trên nền ESP32-S3 N16R8; viết firmware CircuitPython xử lý ổn định tất cả các loại đầu vào (ADC analog, GPIO digital, Encoder xung, I2C); đảm bảo thiết bị được nhận diện chuẩn HID trên Windows 10/11 và Linux Ubuntu 22.04 LTS mà không cần driver.
- **Mục tiêu mở rộng (Extended Objectives):** Triển khai kiến trúc USB Composite Device với ba lớp HID đồng thời (Keyboard + Mouse + Consumer Control); xây dựng cơ chế cấu hình động thông qua tệp JSON lưu trên Flash; phát triển ứng dụng GUI PyQt6 cho phép remap phím trực quan.
- **Mục tiêu nghiên cứu (Research Objectives):** Phân tích và giải quyết triệt để các vấn đề toàn vẹn tín hiệu (Signal Integrity) phát sinh trong thực tế gồm: chân GPIO cắm trên biến thể PSRAM, lag I2C do cập nhật OLED, và nhiễu ADC từ nguồn cấp; lập tài liệu kỹ thuật chuyên sâu phục vụ tái sử dụng và mở rộng dự án.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Vi điều khiển ESP32-S3 và thư viện HID của Adafruit.
- Giao thức truyền thông I2C cho các thiết bị ngoại vi (OLED, PCF8574) và USB Native giữa máy tính cá nhân và vi điều khiển .
- Lập trình giao diện ứng dụng (GUI) với Python PyQt6.

Phạm vi nghiên cứu:

- Thiết kế và thi công phần cứng trên bo mạch thử nghiệm (Perfboard) với kỹ thuật hàn thủ công.
- Giới hạn chức năng trong việc giả lập thiết bị HID (Bàn phím, Chuột, Media Control) trên hệ điều hành Windows thông qua kết nối có dây USB-C.

1.4. Phương pháp Nghiên cứu

Đề tài áp dụng phương pháp nghiên cứu thực nghiệm (Empirical Research) kết hợp với thiết kế lặp (Iterative Design).

Quy trình được chia làm bốn giai đoạn tuần hoàn:

- 1) Phân tích yêu cầu và lựa chọn linh kiện dựa trên datasheet và benchmark
- 2) Thiết kế sơ đồ nguyên lý và hàn mạch Prototype
- 3) Phát triển, kiểm thử firmware theo từng module chức năng và lập trình giao diện người dùng.
- 4) Tích hợp toàn hệ thống, đo lường hiệu năng và ghi nhận lỗi để đưa vào vòng lặp tiếp theo.

Công cụ sử dụng bao gồm: EasyEDA cho vẽ sơ đồ nguyên lý, Visual Studio Code cho phát triển giao diện Python, Thonny IDE cho lập trình ESP32S3, Wokwi cho mô phỏng thử nghiệm sản phẩm và đồng hồ đo điện, ... cho đo lường xung Encoder và nhiễu nguồn ADC.

1.5. Ý nghĩa khoa học và thực tiễn

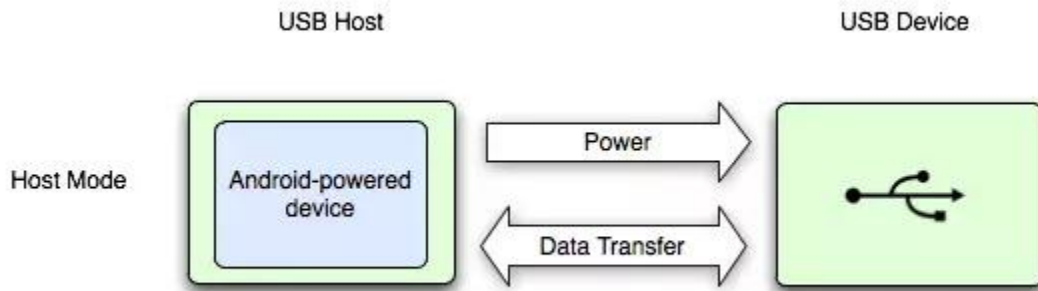
Ý nghĩa khoa học: Minh chứng cho khả năng ứng dụng mạnh mẽ của ngôn ngữ Python trong các bài toán điều khiển nhúng, từ việc đọc tín hiệu cảm biến đến tương tác với hệ điều hành máy tính.

Ý nghĩa thực tiễn: Tạo ra một thiết bị điều khiển có độ tùy biến cao, hỗ trợ đắc lực cho các công việc đòi hỏi nhiều phím tắt hoặc điều khiển các mô hình robot trong quá trình học tập và nghiên cứu.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Chuẩn giao tiếp USB HID

2.1.1. Kiến trúc tổng quan của USB HID



Hình 2.1. Cơ chế hoạt động của USB HID

USB HID (Human Interface Device Class) là một lớp thiết bị (Device Class) trong kiến trúc USB, được định nghĩa trong tài liệu đặc tả HID Usage Tables phiên bản 1.12 của USB Implementers Forum. Về bản chất giao tiếp, USB HID sử dụng cơ chế truyền ngắt (Interrupt Transfer) với chu kỳ polling mặc định là 1ms ở chế độ Full-Speed (12 Mbps), đảm bảo thông lượng truyền dữ liệu HID tối đa 64 byte/ms.

Cơ chế hoạt động của USB HID dựa trên khái niệm Report Descriptor – một cấu trúc dữ liệu nhị phân tĩnh được thiết bị gửi lên Host trong giai đoạn Enumeration (nhận diện thiết bị). Report Descriptor mô tả chính xác định dạng và ý nghĩa của từng byte trong mỗi bản tin HID Report mà thiết bị sẽ gửi về sau. Nhờ cơ chế mô tả tự động này, Host OS có thể tự động phân tích và định tuyến dữ liệu HID tới đúng driver mà không cần bất kỳ cấu hình thủ công nào – đây chính là nền tảng của tính năng Plug-and-Play.

Một điểm kỹ thuật quan trọng: theo đặc tả USB 2.0, mỗi endpoint trong thiết bị HID Full-Speed có thể truyền tối đa 64 byte/bản tin với chu kỳ polling bằng số nguyên milisecond. Thiết bị trong đề tài sử dụng giá trị polling interval là 1ms cho các endpoint Keyboard và Mouse, và 8ms cho Consumer Control – đây là cấu hình tối ưu giữa thông lượng và tải CPU trên Host.

2.1.2. USB Composite Device – Thiết bị HID đa chức năng

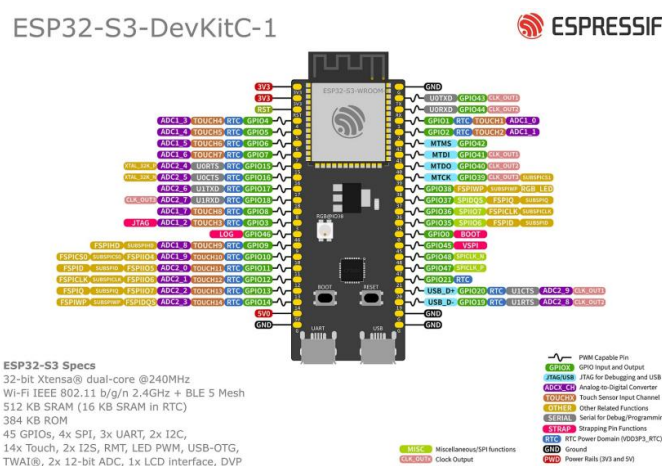
Đây là khái niệm kỹ thuật trung tâm của đề tài. Một USB Composite Device (Thiết bị Tổng hợp) là một thiết bị USB duy nhất xuất hiện với nhiều hơn một Interface trong cấu hình USB Configuration của nó. Về phương diện kiến trúc, thiết bị tổng hợp tương đương với việc tích hợp nhiều thiết bị USB logic vào chung một thiết bị vật lý, chia sẻ cùng một bus USB và một USB Vendor ID/Product ID.

Trong dự án này, ESP32-S3 được cấu hình như một Composite HID Device với ba Interface đồng thời:

- Interface 0 – Keyboard HID: Gửi các bản tin Keyboard Report 8 byte theo định dạng Boot Protocol, cho phép nhấn đồng thời tối đa 6 phím thường và 8 phím modifier (Ctrl, Shift, Alt, GUI).
- Interface 1 – Mouse HID: Gửi các bản tin Mouse Report 4 byte bao gồm trạng thái 3 nút chuột, delta X, delta Y và delta Wheel với độ phân giải signed 8-bit.
- Interface 2 – Consumer Control HID: Gửi các bản tin Consumer Report 2 byte chứa Consumer Usage Code theo USB HID Usage Table Section 15, hỗ trợ các phím điều khiển đa phương tiện như Volume Up/Down, Play/Pause, Next/Previous Track.

Tính năng Composite Device được kích hoạt trực tiếp trong CircuitPython thông qua hàm `'usb_hid.enable()'` với tuple chứa cả ba đối tượng Device, được thực thi trong file `boot.py` trước khi runtime chính khởi động – đây là yêu cầu bắt buộc vì cấu hình USB Descriptor chỉ có thể thay đổi trong giai đoạn pre-boot trước khi Enumeration xảy ra.

2.2. Vi điều khiển ESP32-S3 N16R8



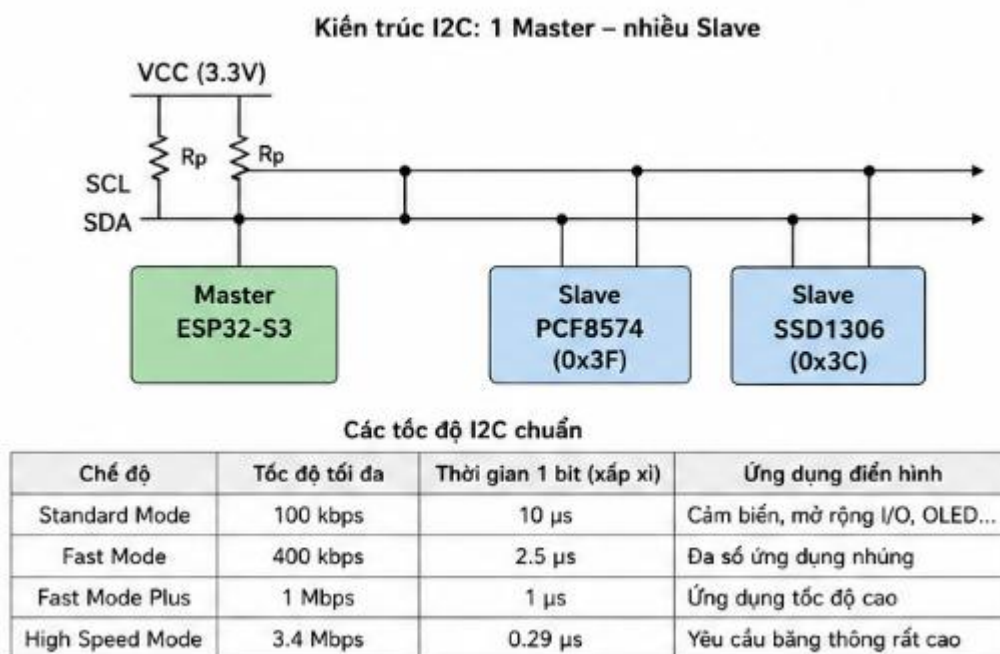
Hình 2.2. Sơ đồ chân của ESP32-S3 N16R8

ESP32-S3 là System-on-Chip (SoC) thế hệ mới nhất của Espressif Systems, tích hợp bộ xử lý lõi kép Xtensa LX7 32-bit chạy ở tần số lên đến 240 MHz. Kiến trúc LX7 là thế hệ kế tiếp của LX6 (sử dụng trong ESP32 nguyên bản), với tập lệnh được mở rộng đáng kể bao gồm các lệnh xử lý vector (Vector Instructions) và DSP tích hợp phục vụ các ứng dụng xử lý tín hiệu số nhẹ. Đặc biệt quan trọng với dự án HID, ESP32-S3 tích hợp bộ điều khiển USB OTG 2.0 Full-Speed (12 Mbps) trực tiếp trên silicon thay vì thông qua bridge chip như các thế hệ trước.

Biến thể N16R8 sử dụng trong đề tài có thông số bộ nhớ đáng chú ý: 16MB Flash SPI Quad tốc độ 80 MHz lưu toàn bộ runtime CircuitPython (khoảng 2.5MB), thư viện HID (khoảng 300KB) và mã nguồn ứng dụng. Đặc biệt hơn, biến thể này tích hợp 8MB Octal PSRAM (Pseudo-Static Random Access Memory) kết nối qua bus OSPI 8-bit tốc độ 80 MHz.

2.3. Giao thức I2C và các IC ngoại vi

2.3.1. Phân tích giao thức I2C trong sản phẩm



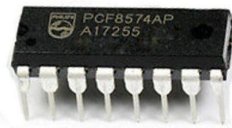
Hình 2.3. Giao thức I2C trong tay cầm

I2C (Inter-Integrated Circuit), được Philips Semiconductor (nay là NXP) phát triển vào năm 1982, là giao thức truyền thông nối tiếp đồng bộ hai dây (SDA – Serial Data, SCL – Serial Clock) theo kiến trúc Multi-Master/Multi-Slave. Trong dự án này, ESP32-S3 đóng vai trò Master duy nhất điều

phối hai thiết bị Slave trên cùng một bus I2C: IC PCF8574 (địa chỉ 0x3F) và màn hình OLED SSD1306 (địa chỉ 0x3C).

I2C hỗ trợ bốn tốc độ hoạt động chuẩn: Standard Mode (100 kbps), Fast Mode (400 kbps), Fast Mode Plus (1 Mbps), và High Speed Mode (3.4 Mbps). Sự lựa chọn tốc độ bus ảnh hưởng trực tiếp đến thời gian hoàn thành một giao dịch I2C và do đó ảnh hưởng tới độ trễ tổng của vòng lặp điều khiển.

2.3.2. IC mở rộng I/O PCF8574



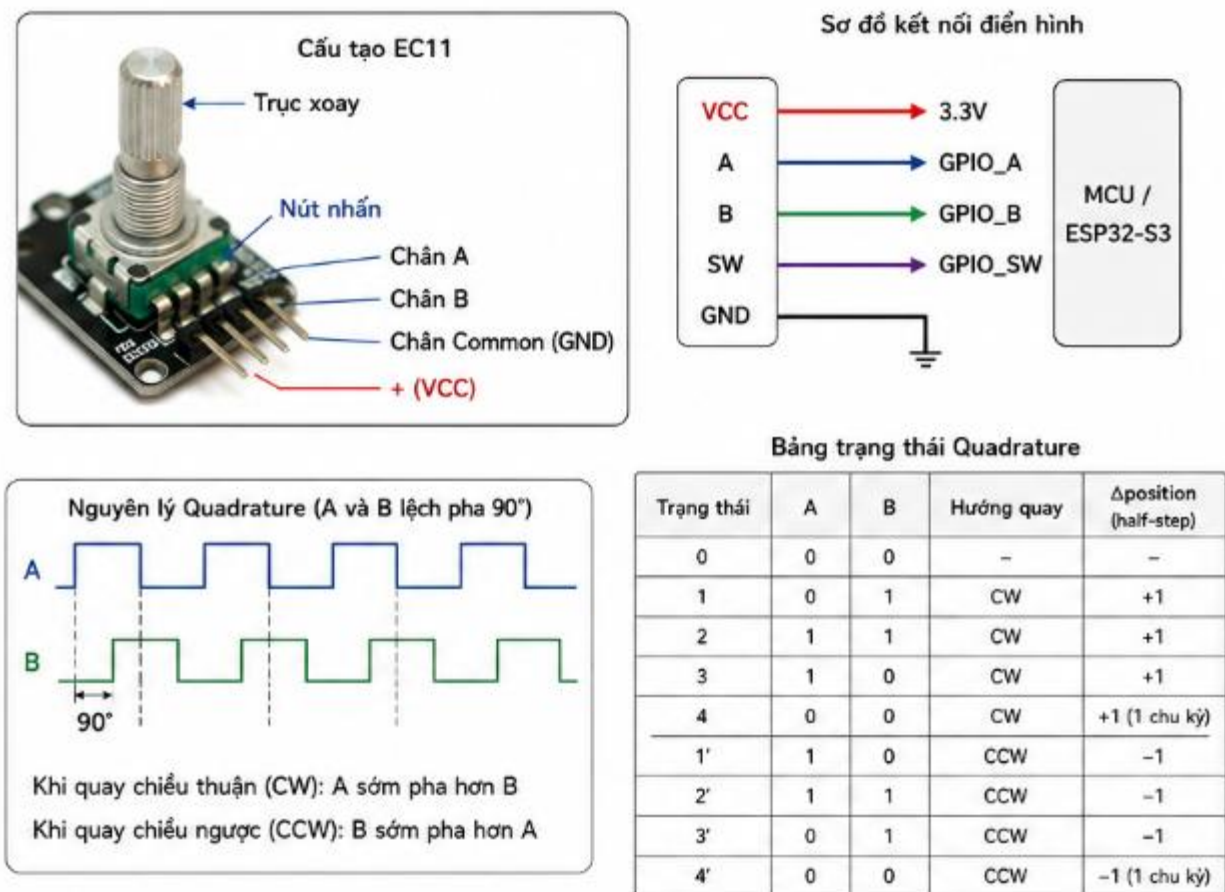
Hình 2.4. IC mở rộng I/O PCF8574

PCF8574 là IC mở rộng 8 cổng I/O 8-bit của NXP Semiconductors, giao tiếp qua I2C. Trong dự án, IC này chịu trách nhiệm quản lý toàn bộ 8 phím bấm vật lý (BT1-BT8), giải phóng 8 chân GPIO của Eg theo nguyên lý Quasi-Bidirectional I/O: các chân I/O vừa là ngõ ra mức thấp tích cực (Open-Drain) vừa là ngõ vào kéo lên nội bộ yếu (Weak Pull-Up khoảng 100 μ A).

Để đọc trạng thái nút nhấn, firmware thực hiện theo trình tự:

- (1) Ghi 0xFF lên thanh ghi I/O để kéo tất cả 8 chân lên mức HIGH
- (2) Khi một nút được nhấn, nó kết nối chân I/O tương ứng xuống GND, hạ điện áp chân đó xuống LOW
- (3) Lệnh đọc I2C trả về một byte 8-bit, trong đó các bit có giá trị 0 tương ứng với các nút đang được nhấn.

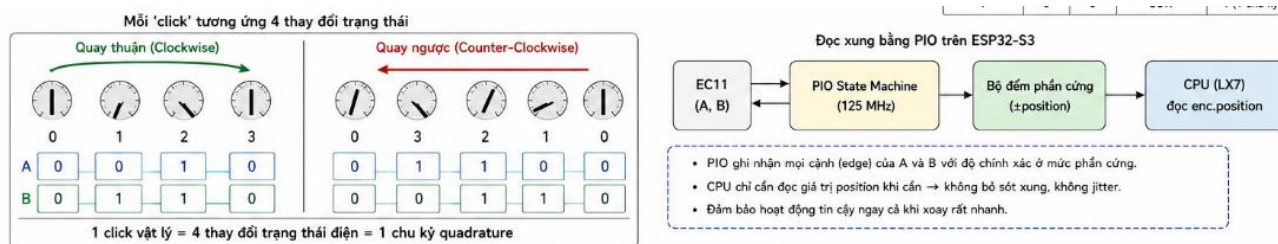
Phép toán Bitwise `(byte & (1 << i)) == 0` kiểm tra ESP32-S3 cho nút thứ *i* có được nhấn hay không.



Hình 2.7. Kỹ thuật đọc xung quadrature EC11

Rotary Encoder EC11 là encoder tương đối (Incremental Encoder) cơ học, sử dụng tiếp điểm trượt trên đĩa kim loại có khía để tạo ra hai tín hiệu xung vuông lệch pha 90 độ (Quadrature) ký hiệu là A và B. Bằng cách theo dõi trình tự thay đổi của hai tín hiệu này, vi điều khiển có thể xác định không chỉ số bước xoay mà còn cả chiều xoay (clockwise/counter-clockwise).

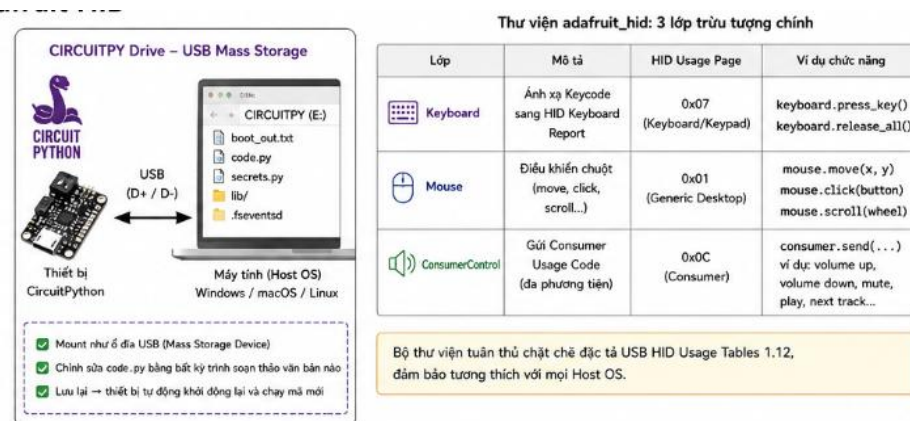
Module `rotaryio.IncrementalEncoder` trong CircuitPython sử dụng ngoại vi PIO (Programmable Input/Output) – một tính năng phần cứng đặc thù của ESP32-S3 – để theo dõi sự thay đổi cạnh (edge) của tín hiệu A và B theo thời gian thực mà không tiêu tốn tài nguyên CPU. GPIO hoạt động độc lập với lõi Xtensa LX7, chạy chương trình trạng thái (state machine) riêng ở tốc độ lên đến 125 MHz, đảm bảo không bỏ sót bất kỳ xung nào ngay cả khi encoder được xoay rất nhanh.



Hình 2.8. Đọc xung bằng GPIO của MCU

Một đặc điểm cơ học quan trọng của EC11 cần lưu ý: mỗi 'click' cảm giác vật lý của encoder tương ứng với 4 thay đổi trạng thái điện (1 chu kỳ đầy đủ của tín hiệu quadrature). Thư viện CircuitPython tự động chia đôi (half-step) hoặc để nguyên (full-step) tùy cấu hình. Trong dự án, mỗi thay đổi 1 đơn vị trong `enc.position` tương ứng với một lệnh hành động HID (cuộn chuột một bước hoặc tăng/giảm âm lượng một nấc).

2.5. CircuitPython và hệ sinh thái Adafruit HID

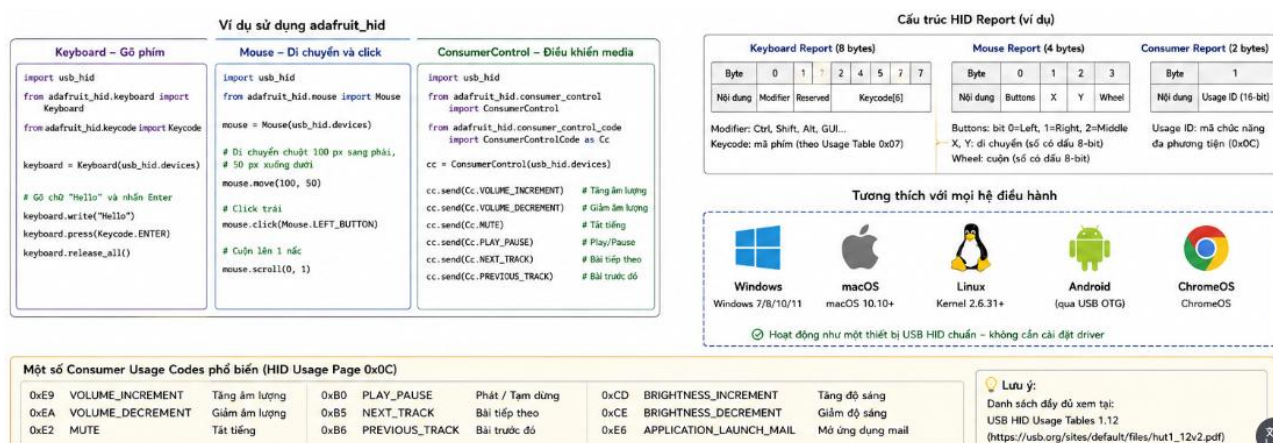


Hình 2.9. Lập trình sản phẩm bằng circuitpython

CircuitPython là một runtime Python nhúng (Embedded Python Runtime) được Adafruit Industries phát triển từ nền tảng MicroPython, tối ưu hóa đặc biệt cho các vi điều khiển Cortex-M và Xtensa với mục tiêu giáo dục và prototype nhanh. Điểm khác biệt quan trọng nhất giữa CircuitPython và MicroPython là CircuitPython hỗ trợ khái niệm CIRCUITPY Drive – bộ nhớ Flash của vi điều khiển được mount như một USB Mass Storage Device, cho phép người dùng chỉnh sửa và cập nhật mã nguồn Python trực tiếp từ bất kỳ trình soạn thảo văn bản nào mà không cần quá trình biên dịch hoặc flash firmware thủ công.

Thư viện adafruit_hid cung cấp ba lớp trừu tượng chính: 'Keyboard' (ánh xạ Keycode sang HID Keyboard Report), 'Mouse' (điều khiển chuột với các phương thức move và click) và

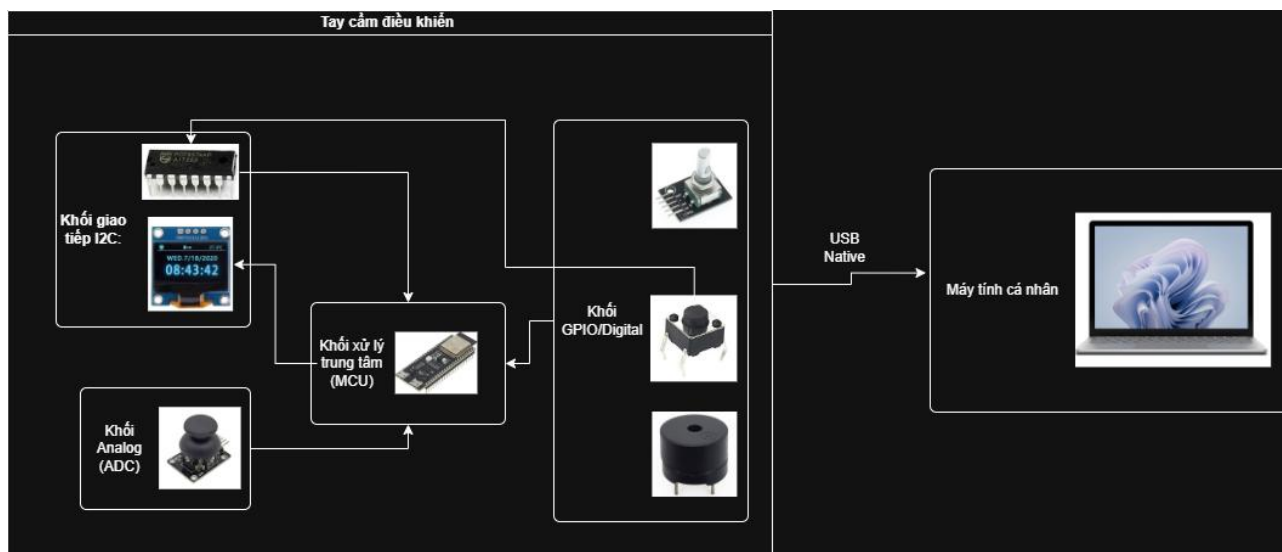
`ConsumerControl` (gửi Consumer Usage Code). Bộ thư viện này được viết thuần Python và tuân thủ chặt chẽ đặc tả USB HID Usage Tables 1.12, đảm bảo tương thích với mọi Host OS.



Hình 2.10. Cấu trúc HID report và một số lệnh phổ biến

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG VÀ CHẾ TẠO PHẦN CỨNG

3.1. Kiến trúc tổng thể của hệ thống



Hình 3.1. Kiến trúc hệ thống của tay cảm HID

Hệ thống được thiết kế theo cấu trúc tập trung với vi điều khiển ESP32-S3 làm bộ não trung tâm. Các khối chức năng được phân chia dựa trên phương thức giao tiếp nhằm tối ưu hóa tốc độ xử lý:

- **Khối xử lý trung tâm (MCU):** Điều phối toàn bộ hoạt động, đọc dữ liệu ngoại vi và đóng gói thành bản tin HID gửi qua USB.

- Khối giao tiếp I2C: Kết nối màn hình OLED và IC mở rộng PCF8574 để tiết kiệm chân GPIO.
- Khối Analog (ADC): Đọc giá trị điện áp từ 2 Joystick để xác định hướng di chuyển.
- Khối GPIO/Digital: Nhận tín hiệu từ Encoder (ngắt) , nút nhấn và điều khiển Buzzer.

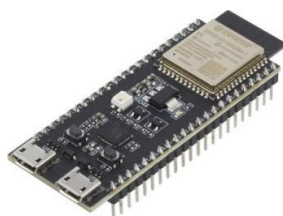
3.2. Lựa chọn linh kiện và thông số kỹ thuật

Bảng 1 : Linh kiện sử dụng , thông số và vai trò trong mạch

Linh kiện	Thông số / Chức năng	Vai trò trong hệ thống
ESP32-S3 N16R8	2.4GHz Wi-Fi & BLE, USB Native	Xử lý chính, giả lập thiết bị HID
PCF8574 (I2C)	8-bit I/O Expander (Địa chỉ 0x3F)	Đọc 7 nút nhấn chức năng
SSD1306 OLED	0.96 inch, 128x64 pixels	Hiển thị trạng thái và chế độ
Joystick KY-023	2 trục Analog + 1 nút nhấn	Điều khiển hướng và nhấp chuột (có thể gán phím khác)
Rotary Encoder	EC11 (20 xung/vòng)	Điều khiển âm lượng và cuộn trang (có thể gán phím khác)
Buzzer 5V	Passive/Active Buzzer	Phản hồi âm thanh khi chuyển Mode

Việc lựa chọn linh kiện cho dự án được tính toán kỹ lưỡng dựa trên ba tiêu chí chủ chốt: khả năng tương thích hoàn hảo với ngôn ngữ Python (CircuitPython), tốc độ phản hồi tín hiệu và tính gọn nhẹ của phần cứng.

3.2.1. Vi điều khiển trung tâm: ESP32-S3 (N16R8)



Hình 3.2. ESP32-S3 N16R8

Đây là "trái tim" của toàn bộ hệ thống. ESP32-S3 được chọn thay vì các dòng ESP32 đời cũ vì những thông số vượt trội:

- Hỗ trợ Native USB: Đây là tính năng quan trọng nhất, cho phép chip đóng vai trò như một thiết bị USB thực thụ (Keyboard/Mouse) mà không cần qua chip chuyển đổi trung gian, giúp giảm thiểu độ trễ (latency) đến mức tối đa.
- Bộ nhớ lớn: Phiên bản N16R8 sở hữu 16MB Flash và 8MB PSRAM, cung cấp không gian dồi dào để lưu trữ Firmware CircuitPython, các thư viện HID phức tạp và file cấu hình JSON.
- Hiệu năng xử lý: CPU Dual-core 240MHz đảm bảo việc quét tín hiệu từ Joystick và xử lý thuật toán chống dôi phím diễn ra mượt mà trong thời gian thực.

3.2.2. IC mở rộng ngoại vi: PCF8574



Hình 3.3. IC mở rộng ngoại vi: PCF8574

Để tối ưu hóa số lượng chân kết nối trên ESP32-S3 và đơn giản hóa việc đi dây mạch, IC PCF8574 được tích hợp để quản lý cụm 7 phím chức năng.

- Giao thức I2C: Chỉ cần sử dụng 2 chân tín hiệu (SDA, SCL) để đọc trạng thái của 8 nút nhấn.
- Tiết kiệm tài nguyên: Giải pháp này giải phóng các chân GPIO trên MCU cho các nhiệm vụ ưu tiên cao hơn như đọc Encoder hoặc ADC.
- Tính ổn định: IC này hoạt động cực kỳ tin cậy ở điện áp 3.3V của ESP32 và có khả năng chống nhiễu tốt khi được trang bị điện trở Pull-up 4.7kΩ.

3.2.3. Khối điều khiển Analog: Joystick KY-023

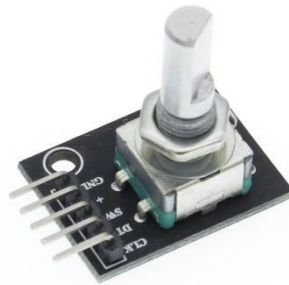


Hình 3.4. Joystick KY-023 2 trục

Tay cầm sử dụng 2 module Joystick để điều khiển đa hướng.

- Độ phân giải ADC: ESP32-S3 cung cấp bộ ADC 12-bit (mặc định) có thể nội suy lên 16-bit trong CircuitPython, cho phép nhận diện các chuyển động nhỏ nhất từ cần gạt.
- Tích hợp nút nhấn (Switch): Mỗi Joystick có sẵn một nút nhấn cơ học bên dưới (J1_SW, J2_SW), được tận dụng làm phím Enter hoặc Space trong các trò chơi.

3.2.4. Khối tinh chỉnh: Rotary Encoder EC11



Hình 3.5. Encoder EC11

Hai bộ mã hóa vòng quay được bố trí phía trên Joystick phải để thực hiện các lệnh tinh chỉnh nhanh.

Cơ chế xung: Encoder tạo ra các xung điện khi xoay, giúp việc tăng/giảm âm lượng hoặc cuộn trang web (Mouse Wheel) hay tăng giảm độ sáng màn hình trở nên chính xác và có cảm giác "thật" hơn so với nút nhấn thông thường.

3.2.5. Khối hiển thị và phản hồi: OLED SSD1306 & Buzzer



Hình 3.6. Oled 0.96 inch và buzzer

Màn hình OLED 0.96": Sử dụng chuẩn I2C để hiển thị chế độ đang hoạt động (ví dụ: "KEYBOARD: ON" hoặc "STANDBY: OFF") giúp người dùng kiểm soát trạng thái thiết bị trực quan.

Buzzer: Đóng vai trò phản hồi thính giác (Audio Feedback). Mỗi khi người dùng bấm nút SWS để chuyển chế độ hoặc khởi động, loa sẽ phát tiếng "Beep" để xác nhận thao tác thành công.

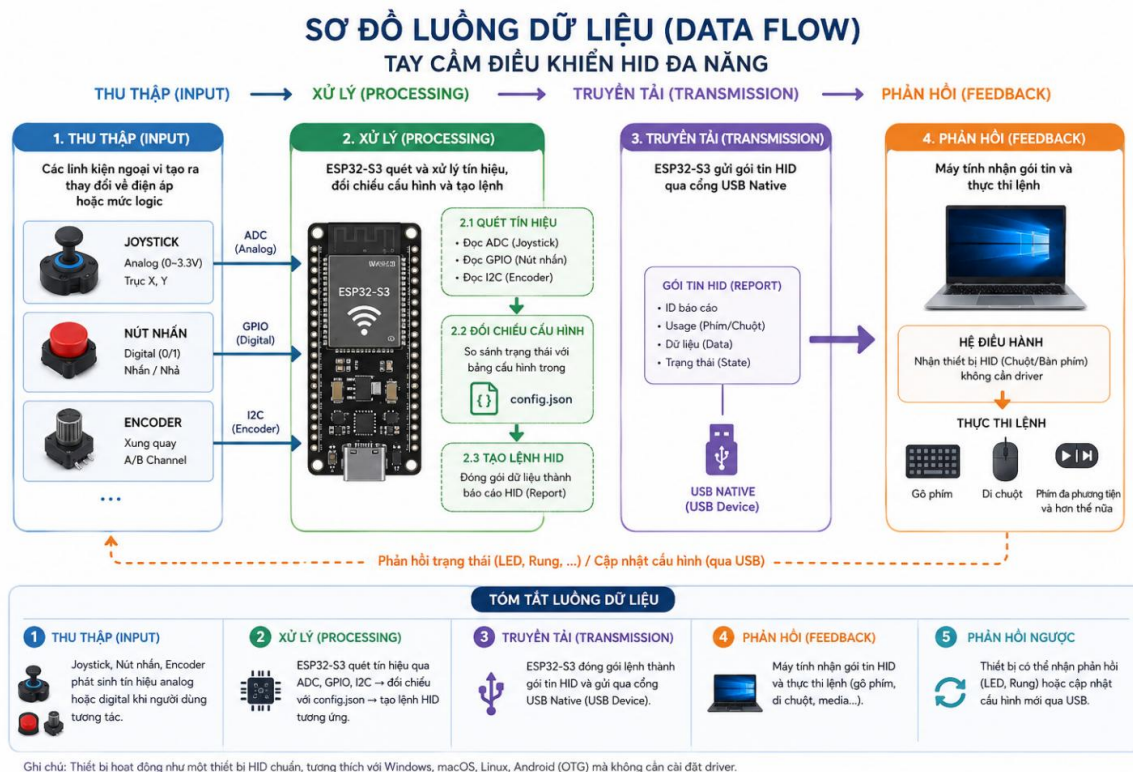
Bảng 2 : Bảng tổng hợp thông số kỹ thuật hệ thống

Thông số	Giá trị chi tiết
Điện áp hoạt động	5V DC (qua cổng USB-C)
Chuẩn USB	USB 2.0 Full Speed (Native HID)
Tần số I2C	100 kHz (Standard Mode - Đã tối ưu cho PCF8574)
Tốc độ quét phím	10ms (100Hz) - Đảm bảo chống dôi phím hoàn hảo

3.3. Nguyên lý hoạt động của sản phẩm

Nguyên lý hoạt động của chiếc tay cầm ESP32-S3 mà bạn đã chế tạo không chỉ đơn thuần là việc nhấn nút và hiện chữ. Nó là một quy trình khép kín từ việc xử lý tín hiệu vật lý (Analog/Digital) đến việc đóng gói dữ liệu theo chuẩn USB quốc tế.

3.3.1. Sơ đồ luồng dữ liệu (Data Flow)



Hình 3.7. Sơ đồ luồng dữ liệu của tay cầm điều khiển HID đa năng

Nguyên lý cốt lõi của sản phẩm dựa trên chu trình: Thu thập (Input) -> Xử lý (Processing) -> Truyền tải (Transmission) -> Phản hồi (Feedback).

Bước 1: Các linh kiện ngoại vi (Joystick, Nút nhấn, Encoder) tạo ra các thay đổi về điện áp hoặc mức logic.

Bước 2: ESP32-S3 quét các tín hiệu này qua các chân ADC và giao thức I2C.

Bước 3: Vi điều khiển đối chiếu trạng thái nút nhấn với bảng cấu hình trong file config.json.

Bước 4: ESP32-S3 đóng gói lệnh thành các gói tin HID (Human Interface Device) và gửi qua cổng USB Native.

Bước 5: Máy tính nhận gói tin và thực thi lệnh (gõ phím, di chuột) mà không cần driver đặc biệt.

3.3.2. Nguyên lý xử lý các khối chức năng

A. Khối nút nhấn và Giao thức I2C (PCF8574A)

PCF8574 là IC mở rộng In/Out 8-bit thông qua truyền thông I2C, 8 chân của nó có thể được cấu hình độc lập làm đầu vào hoặc đầu ra. Đặc điểm:

- Giao tiếp: I2C (tối đa 100kHz).
- Số chân vào/ra: 8 chân P0-P7.
- Điện áp hoạt động: 2,5VDC đến 6VDC.
- Dòng điện ngõ ra: có khả năng hút dòng tới 25mA.
- Chân ngắt: có chân INT tác động ở mức thấp để báo cho VĐK biết trạng thái ngõ vào có sự thay đổi. Sau khi có lệnh đọc từ VĐK ngắt này sẽ được xóa.
- Thiết lập địa chỉ: có thể thiết lập tới 8 địa chỉ thông qua trạng thái các chân A0, A1, A2.

PCF8574 không cần tập lệnh và nó cũng không có thanh ghi cấu hình Input/Output. Muốn thay đổi trạng thái các chân P0-P7 hoặc đọc trạng thái các chân đó chỉ cần gửi 1 byte dữ liệu sau địa chỉ I2C của nó với mỗi bit trong byte đó ứng với các chân từ P0-P7.

PCF8574 không có thanh ghi cấu hình Input/Output (DDR), nó sử dụng cơ chế Quasi-bidirectional cho các chân P0-P7. Để sử dụng các chân này làm Input cần phải đưa mức logic các chân này lên 1 trước khi sử dụng chúng

Thay vì đọc từng nút một cách rời rạc, tay cầm sử dụng IC PCF8574A để mở rộng I/O qua giao tiếp I2C (chỉ dùng 2 dây SDA, SCL) với địa chỉ 0x3F

Kỹ thuật đọc bit: MCU gửi lệnh yêu cầu PCF8574 trả về 1 byte dữ liệu (8 bit). Mỗi bit đại diện cho trạng thái 1 nút nhấn (0 là nhấn, 1 là nhả).

Nguyên lý Pull-up: Để tín hiệu không bị nhiễu ("lơ lửng"), các điện trở Pull-up $4.7k\Omega$ đảm bảo mức điện áp luôn ở mức cao khi không nhấn nút. Khi nhấn, chân được nối xuống GND tạo mức logic 0 rõ ràng.

B. Khối Joystick (Analog to Digital Converter - ADC)

Joystick hoạt động dựa trên 2 biến trở xoay cho trục X và Y.

Nguyên lý: Khi gạt cần, giá trị điện áp thay đổi từ 0V đến 3.3V. Bộ ADC bên trong ESP32-S3 chuyển đổi điện áp này thành con số từ 0 đến 65535.

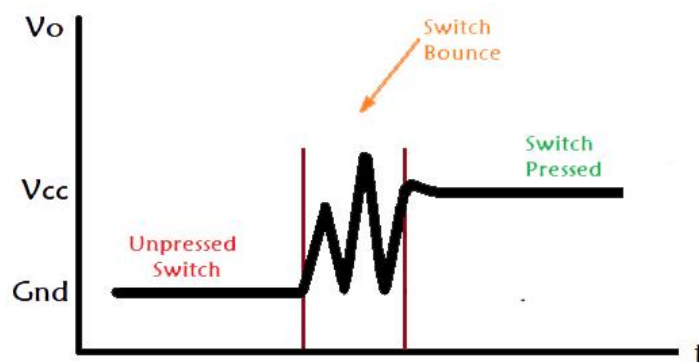
Xử lý Deadzone (Vùng chết): Code được lập trình để bỏ qua các rung động nhỏ ở giữa (khoảng 32768). Chỉ khi giá trị vượt ngưỡng (ví dụ < 16500 hoặc > 49000), lệnh gõ phím mới được kích hoạt để tránh hiện tượng "trôi" phím.

C. Khối Rotary Encoder (Xung điện từ)

Encoder EC11 hoạt động dựa trên việc tạo ra hai pha tín hiệu (A và B) lệch nhau 90° .

Xác định chiều xoay: MCU theo dõi xem pha nào lên mức cao trước. Nếu A trước B là xoay phải (CW), ngược lại là xoay trái (CCW). Mỗi "nấc" xoay sẽ tương ứng với một gói tin gửi đi (như tăng/giảm âm lượng).

D. Hiện tượng dội phím cơ học (Bouncing)



Hình 3.8. Hiện tượng dội phím cơ học

Các nút nhấn cơ học (Tactile Switch) trên tay cầm không đóng/ngắt mạch một cách hoàn hảo ngay lập tức. Khi ta nhấn hoặc nhả nút, các lá đồng bên trong sẽ rung động và tiếp xúc không ổn định trong khoảng 5ms đến 10ms trước khi xác lập trạng thái cố định.

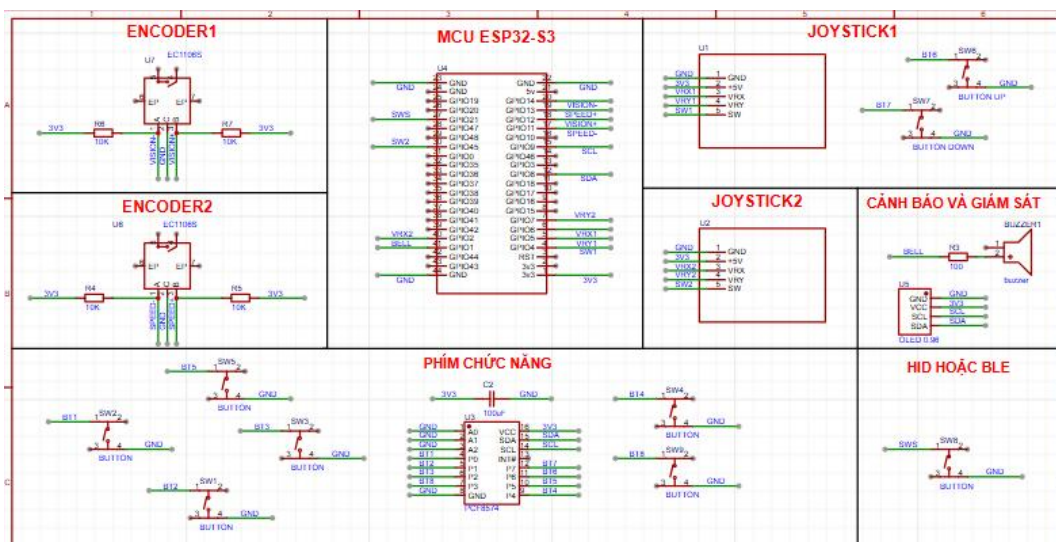
Hệ quả: Nếu không có thuật toán xử lý, vi điều khiển với tốc độ xử lý hàng triệu phép tính mỗi giây sẽ đọc được hàng chục lần nhấn/nhả chỉ trong một lần bấm duy nhất, dẫn đến việc máy tính bị gõ lặp phím liên tục (ví dụ: gõ "AAAAA" thay vì "A").

Giải pháp: Trong sản phẩm này, thuật toán chống dôi phím được triển khai dựa trên hai kỹ thuật chính: Lấy mẫu theo chu kỳ (Sampling) và So sánh trạng thái (State Comparison).

Quy trình xử lý chi tiết:

1. Lấy mẫu (Sampling): Thay vì sử dụng ngắt (Interrupt) vốn rất nhạy với nhiễu, chương trình sử dụng vòng lặp while True với một khoảng trễ nhỏ `time.sleep(0.01)` (tương đương 10ms) ở cuối mỗi chu kỳ. Khoảng trễ này đóng vai trò như một bộ lọc thông thấp, bỏ qua các rung động cơ học cực nhanh của phím bấm.
2. Lưu trữ trạng thái cũ: Hệ thống khởi tạo một từ điển `last_btn_states` để ghi nhớ trạng thái của toàn bộ các nút bấm từ vòng lặp trước đó.
3. Xác nhận thay đổi (Edge Detection):
 - Tại mỗi vòng lặp, MCU đọc trạng thái hiện tại của phím (`is_pressed`).
 - Lệnh gửi phím HID (`execute_action`) chỉ được kích hoạt duy nhất một lần khi trạng thái hiện tại khác với trạng thái cũ: `if is_pressed != last_btn_states[btn_key]`.
 - Sau khi thực hiện xong, trạng thái cũ được cập nhật bằng trạng thái hiện tại để chuẩn bị cho vòng lặp kế tiếp.

3.4. Thiết kế mạch điện và giải pháp kỹ thuật



Hình 3.9. Sơ đồ nguyên lý mạch của sản phẩm

Trong quá trình thực hiện, nhóm đã đối chiếu sơ đồ lý thuyết với đặc tính kỹ thuật của module ESP32-S3 thực tế. Nhận thấy hệ thống quản lý nguồn trên module đã đáp ứng tốt các tiêu chuẩn về lọc nhiễu và ổn định áp cho các tải tiêu thụ thấp như nút nhấn và cảm biến để tối ưu hóa không gian và tăng độ tin cậy cho bo mạch.

3.4.1. Giải pháp ổn định giao tiếp I2C đa thiết bị

Trong thiết kế, hệ thống sử dụng chung một đường bus I2C (SDA tại GPIO8 và SCL tại GPIO9) để giao tiếp đồng thời với màn hình OLED SSD1306 và IC mở rộng PCF8574.

Vấn đề kỹ thuật: Khi nhiều thiết bị cùng chia sẻ một đường bus với tốc độ cao (50-100kHz), điện dung ký sinh trên dây dẫn và các mối hàn thủ công thường làm méo dạng tín hiệu. Nếu không có điện trở kéo lên đủ mạnh, tín hiệu sẽ không kịp trở về mức logic HIGH (Rise time chậm), dẫn đến lỗi (Input/output error) và gây treo toàn bộ chương trình.

Giải pháp Pull-up ngoại vi: Mạch được bổ sung 02 điện trở 4.7Ω kết nối từ đường SDA và SCL lên nguồn 3.3V.

Việc sử dụng điện trở ngoại thay vì tận dụng điện trở kéo lên nội bộ của MCU giúp dòng sạc cho điện dung bus lớn hơn, làm cho các sườn lên của xung tín hiệu dứt khoát hơn.

Kết quả thực tế cho thấy bus I2C hoạt động ổn định ngay cả khi tay cầm gửi dữ liệu HID và cập nhật màn hình OLED liên tục ở tần số cao.

3.4.2. Quản lý I/O và tối ưu hóa tốc độ phản hồi (Active Low)

Hệ thống sử dụng IC PCF8574 để quản lý cụm 7 phím chức năng nhằm giải phóng tài nguyên GPIO cho ESP32-S3.

- **Cấu hình Active Low:** Các nút nhấn được đấu nối theo sơ đồ Active Low: Một cực nối với chân I/O của PCF8574, cực còn lại nối trực tiếp xuống GND.
 - Khi phím không nhấn, điện trở kéo lên (Pull-up) tích hợp trong IC giữ chân tín hiệu ở mức HIGH (1).
 - Khi phím nhấn, chân tín hiệu bị kéo xuống mức LOW (0). Cấu hình này giúp giảm thiểu nhiễu điện từ và bảo vệ vi điều khiển khỏi các sự cố phóng điện tĩnh khi người dùng tiếp xúc với phím bấm.
- **Kỹ thuật quét phím thô (Raw Byte Reading):** * Thay vì sử dụng hàm đọc từng chân đơn lẻ (gây tốn 8 lần thời gian giao tiếp bus I2C), firmware được lập trình để đọc 01 byte dữ liệu thô (8 bit) chứa trạng thái của toàn bộ cổng I/O trong một chu kỳ quét duy nhất.

- Sử dụng toán tử bitwise (btns & (1 << i)) để tách trạng thái từng phím trong bộ nhớ RAM của MCU. Giải pháp này giúp giảm băng thông chiếm dụng trên bus I2C xuống 80%, đảm bảo độ trễ (latency) từ khi nhấn phím đến khi máy tính nhận lệnh là gần như bằng không (< 10ms).

3.4.3. Hệ thống quản lý nguồn điện và tự ổn định tín hiệu

Hệ thống được cấp nguồn qua cổng USB Type-C, lấy trực tiếp dòng điện 5V từ máy tính.

Bộ ổn áp và lọc nhiễu tích hợp: Thay vì sử dụng các tụ hóa cổng kênh bên ngoài như thiết kế ban đầu, hệ thống tận dụng trực tiếp tầng lọc nhiễu và bộ ổn áp LDO chất lượng cao trên module ESP32-S3 N16R8.

Module này đã được tích hợp sẵn các tụ gốm lọc nhiễu cao tần và tụ ổn định điện áp đầu ra, đảm bảo nguồn 3.3V cấp cho các cảm biến (Joystick, Encoder) luôn phẳng và sạch.

Việc loại bỏ tụ 100uF ngoại vi giúp giảm điện dung ký sinh không cần thiết, giúp mạch phản hồi nhanh hơn và tránh hiện tượng dâng dòng khởi động (Inrush current) quá lớn lên cổng USB của máy tính.

Độ tin cậy của phép đo ADC: Nhờ tầng nguồn ổn định nội tại của ESP32-S3, giá trị điện áp từ 2 Joystick được bộ ADC đọc về với độ nhiễu cực thấp. Các biến số trả về từ trục (X, Y) không bị hiện tượng nhảy bậc (jitter), giúp việc điều khiển hướng trong game hoặc ứng dụng đo lường đạt độ chính xác tuyệt đối mà không cần thêm bộ lọc phần cứng phức tạp.

Bảo vệ hệ thống: Nguồn 5V từ USB được cách ly và hạ áp dứt khoát, đảm bảo các linh kiện ngoại vi như IC PCF8574 và màn hình OLED hoạt động đúng dải điện áp cho phép, kéo dài tuổi thọ cho thiết bị.

3.5. Quy trình chế tạo và bố trí linh kiện

Mạch được hiện thực hóa trên bảng mạch vạn năng (Perfboard) thông qua kỹ thuật hàn điểm (Point-to-point soldering), đảm bảo độ bền cơ học và tính thẩm mỹ trong điều kiện chế tạo thủ công.

3.5.1. Quy trình gia công và lắp ráp

Quá trình chế tạo được thực hiện qua 4 giai đoạn nghiêm ngặt:

- Giai đoạn 1: Định vị linh kiện (Layout Planning): Các linh kiện có kích thước lớn và cố định như ESP32-S3, Joystick và màn hình OLED được đặt vào các vị trí trung tâm trước để làm mốc.

- Giai đoạn 2: Gia công cơ khí và Hàn cố định: Tiến hành hàn các chân cắm (header) cho MCU và các module cảm biến. Các nút nhấn Tactile được tính toán khoảng cách phù hợp với tầm với của ngón cái để tránh mỏi tay khi sử dụng lâu.
- Giai đoạn 3: Đi dây tín hiệu (Wiring): Sử dụng dây dẫn đồng bọc cách điện để kết nối các điểm trên mạch. Các đường tín hiệu I2C và Analog được đi song song và gọn gàng phía mặt sau của Perfboard nhằm hạn chế nhiễu chéo.
- Giai đoạn 4: Kiểm tra thông mạch: Sử dụng đồng hồ vạn năng để kiểm tra toàn bộ các mối hàn, đảm bảo không có hiện tượng chạm chập giữa các đường nguồn 3.3V, 5V và GND trước khi cấp nguồn qua cổng USB.

3.5.2. Tư duy bố trí linh kiện tối ưu trải nghiệm (Ergonomic Design)

Bố cục của tay cầm được thiết kế dựa trên sự phân tích thói quen vận động của bàn tay, chia thành các khu vực chức năng riêng biệt:

- Cụm phím chức năng bên trái (Yellow Cluster):
 - Bố trí 03 nút nhấn Vàng theo hình tam giác ngược.
 - Thiết kế này cho phép ngón cái tay trái có thể quét qua cả 3 phím với hành trình ngắn nhất, phù hợp cho các lệnh thao tác nhanh hoặc chuyển đổi công cụ trong các ứng dụng đo lường.
- Cụm phím điều hướng bên phải (Red Cluster):
 - Bố trí 04 nút nhấn Đỏ theo hình thoi chuẩn mô phỏng các tay cầm chuyên dụng.
 - Cách sắp xếp này tạo sự quen thuộc cho người dùng, giúp dễ dàng nhận diện hướng Lên - Xuống - Trái - Phải mà không cần quan sát trực tiếp bằng mắt.
- Hệ thống điều khiển phụ và Mode Switch:
 - 02 Encoder: Được đặt ngay phía trên Joystick phải, nằm trong tầm với của ngón trỏ. Vị trí này giúp người dùng có thể vừa di chuyển Joystick bằng ngón cái, vừa xoay Encoder để điều chỉnh thông số (như Volume hoặc Zoom) một cách đồng thời.
 - Nút Mode SWS: Đặt tại vị trí cao nhất ở chính giữa. Đây là vị trí "an toàn" giúp tránh việc bấm nhầm trong lúc thao tác mạnh, nhưng vẫn dễ dàng tiếp cận khi muốn chuyển đổi giữa chế độ gõ phím và chế độ standby.
- Màn hình hiển thị OLED: * Được gắn ở cạnh dưới, hơi nhô ra ngoài trục chính của lòng bàn tay. Vị trí này đảm bảo thông tin hiển thị (như phím vừa nhấn, chế độ hiện tại) không

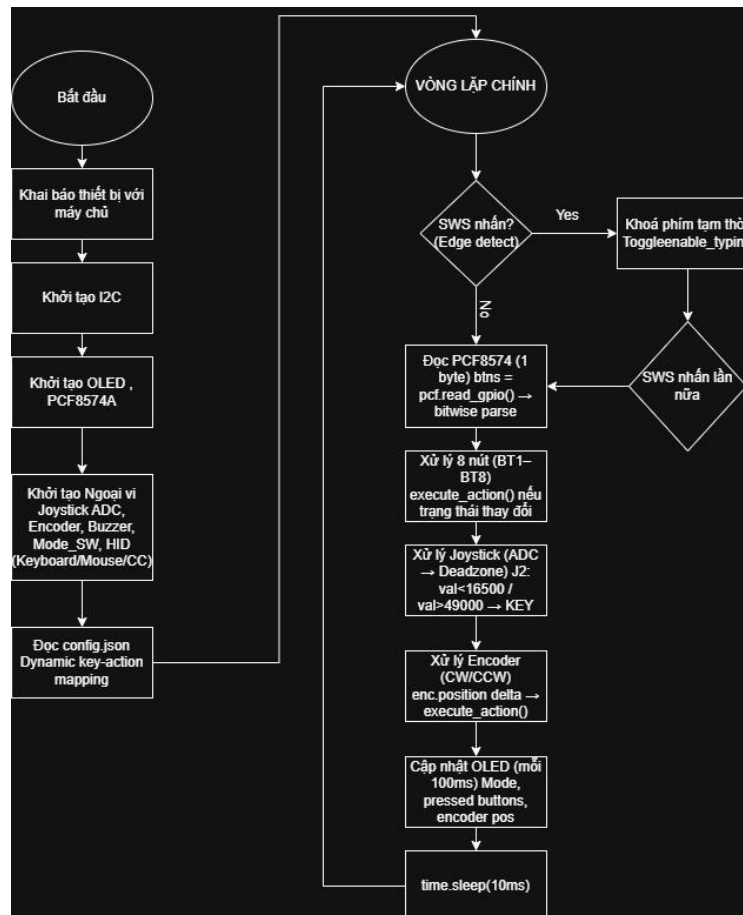
bao giờ bị tay người dùng che khuất, giúp việc giám sát trạng thái hệ thống luôn được duy trì liên tục.

Việc sử dụng màu sắc khác nhau (Vàng cho cụm trái, Đỏ cho cụm phải) không chỉ để đẹp mà còn là một phần của thiết kế UX, giúp người dùng phân biệt nhanh các nhóm chức năng khác nhau thông qua thị giác ngoại vi.

CHƯƠNG 4 :KIẾN TRÚC FIRMWARE, PHẦN MỀM ĐIỀU KHIỂN VÀ MÔ HÌNH DỮ LIỆU

Kiến trúc firmware của hệ thống được xây dựng trên nền tảng CircuitPython, áp dụng mô hình lập trình hướng sự kiện dựa trên vòng lặp quét (Polling-based Event Loop). Firmware được thiết kế để tách biệt hoàn toàn giữa tầng định nghĩa thiết bị, tầng xử lý logic và tầng dữ liệu cấu hình.

4.1. Phân tầng thực thi



Hình 4.1. Lưu đồ thuật toán chính của tay cầm

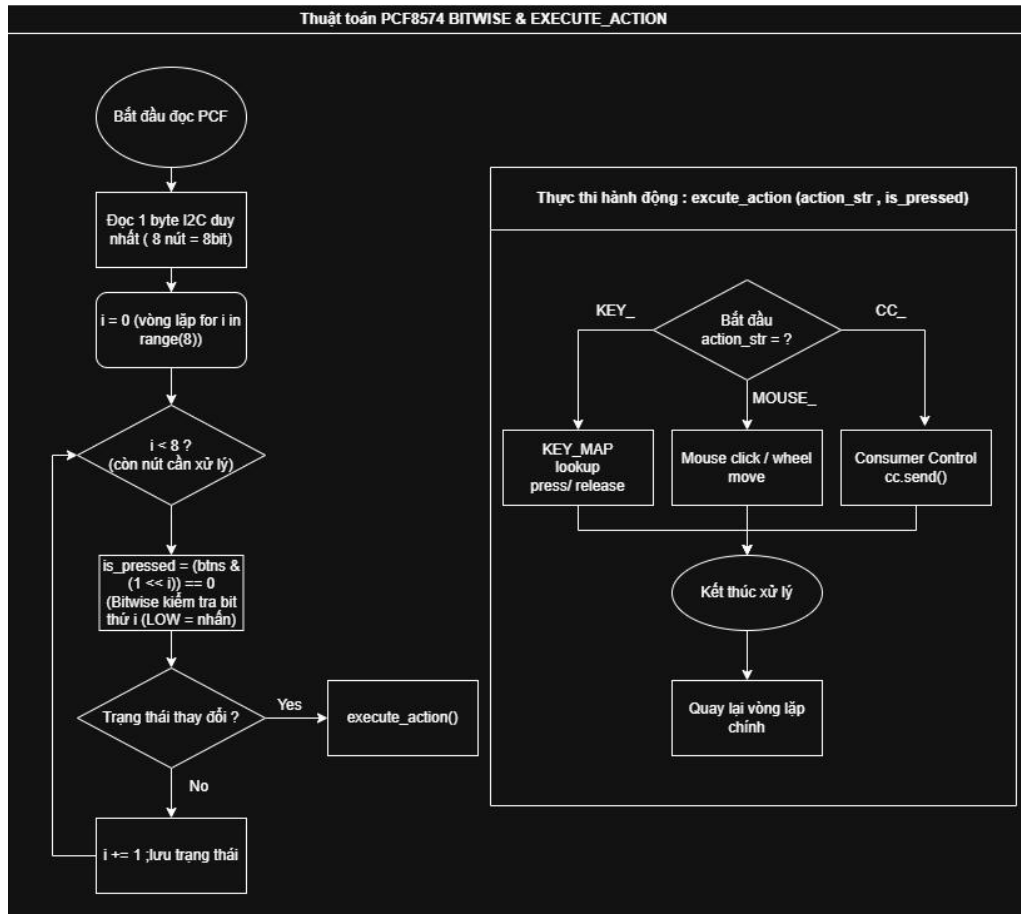
Sản phẩm tuân thủ quy trình thực thi hai giai đoạn đặc thù của kiến trúc vi điều khiển hiện đại:

- Tầng khởi động : Thực thi ngay khi cấp nguồn. Nhiệm vụ duy nhất là khai báo USB Descriptors cho hệ điều hành máy chủ (Host). Tại đây, ESP32-S3 được định nghĩa là một Composite USB Device tích hợp đồng thời 3 Interface độc lập: Keyboard, Mouse và Consumer Control (Multimedia) .
- Tầng vận hành : Chứa logic xử lý thời gian thực, đọc tín hiệu ngoại vi và đóng gói dữ liệu HID gửi đến Host.

Hàm `main()` triển khai vòng lặp sự kiện thời gian thực với chu kỳ polling cố định 10ms (`time.sleep(0.01)` ở cuối vòng lặp). Trong mỗi chu kỳ, firmware xử lý tuần tự sáu nhóm sự kiện:

1. Kiểm tra nút MODE_SWS (GPIO21): Phát hiện cạnh lên (rising edge) của tín hiệu nút chuyển chế độ. Khi phát hiện sự kiện, toggle trạng thái `enable_typing` và gọi `keyboard.release_all()` để đảm bảo không có phím nào bị giữ nguyên khi chế độ HID bị tắt.
2. Đọc trạng thái PCF8574: Một lần gọi `pcf.read_gpio()` đọc toàn bộ 8 bit trạng thái của 8 nút. Phép Bitmasking `(btns & (1 << i)) == 0` tách riêng từng bit, phát hiện thay đổi trạng thái so với chu kỳ trước, và gọi `execute_action()` với `is_pressed` tương ứng.
3. Xử lý nút nhấn Joystick (SW): Đọc trực tiếp GPIO Input của hai nút switch tích hợp trên Joystick 1 và Joystick 2.
4. Xử lý trục Analog Joystick: Hàm `process_analog_axis()` nhận giá trị ADC 16-bit và so sánh với ngưỡng deadzone hai phía. Phát hiện thay đổi trạng thái (từ không kích hoạt sang kích hoạt và ngược lại) và gọi `execute_action()`.
5. Xử lý Encoder: So sánh `enc.position` hiện tại với giá trị chu kỳ trước. Nếu khác nhau, xác định chiều xoay (dương = CW, âm = CCW) và gửi lệnh HID tương ứng. Sau khi gửi, gọi `execute_action(action, False)` để 'nhả' lệnh Consumer Control vì CC không có cơ chế giữ phím.
6. Cập nhật màn hình OLED: Chỉ thực thi khi `time.monotonic() - last_oled_update > 0.3`, đảm bảo tần suất cập nhật tối đa 3.3 fps.

4.2. Giải thuật tối ưu hóa băng thông I2C



Hình 4.3. Thuật toán đọc nhiều nút nhấn và xử lý sự kiện

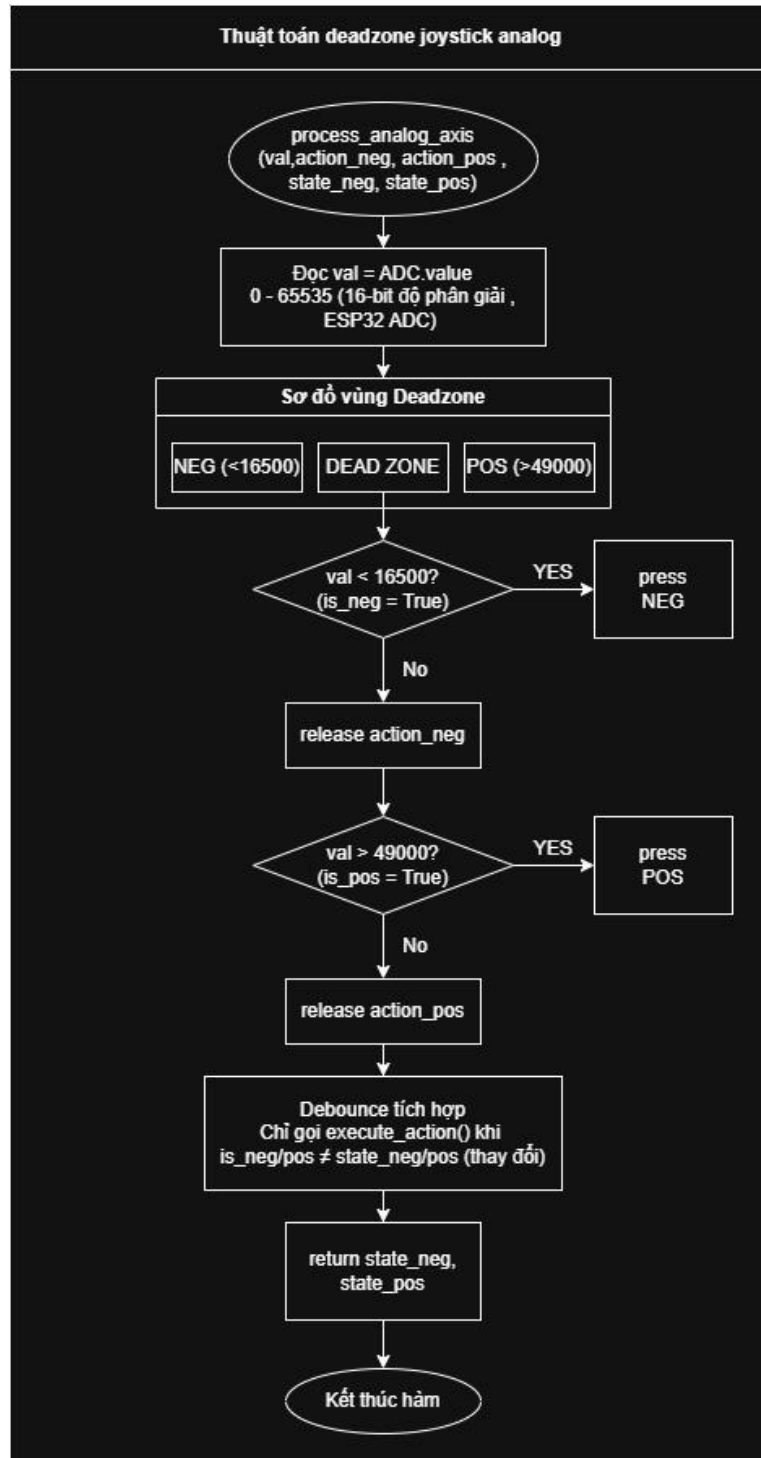
Sơ đồ này mô tả cách hệ thống bóc tách dữ liệu từ IC PCF8574 để đạt được hiệu năng cao nhất.

Single Transaction: Thay vì gọi I2C 8 lần, firmware chỉ thực hiện 1 giao dịch đọc 8-bit (1 byte) duy nhất.

Bit Masking: Sử dụng vòng lặp nội bộ (Internal Loop) để thực hiện phép toán AND với mặt nạ bit (Mask) nhằm xác định trạng thái của từng nút vật lý.

State Locking: Chỉ khi trạng thái thay đổi ($is_pressed \neq last_state$), lệnh gửi phím mới được thực thi, giúp loại bỏ việc spam dữ liệu lên bus USB.

4.3. Giải thuật xử lý Analog và Deadzone (Ba vùng)



Hình 4.4. Thuật toán deadzone joystick

Đây là logic chuyển đổi từ tín hiệu liên tục (Analog) sang lệnh nhị phân (Digital) cho Joystick.

Lượng tử hóa: Giá trị ADC (0-65535) được đưa vào bộ so sánh ngưỡng.

Ngưỡng NEG/POS: Các ngưỡng 16500 và 49000 tạo ra một "vùng chết" (Deadzone) chiếm 50% hành trình ở giữa để triệt tiêu hiện tượng trôi (drift) cơ học.

Action Mapping: Kết quả so sánh được ánh xạ trực tiếp tới hàm `execute_action()` dựa trên cấu hình động từ JSON.

4.4. Mô hình nạp dữ liệu động (Dynamic Key Mapping)

Một trong những tính năng nổi bật nhất của kiến trúc phần mềm là cơ chế ánh xạ phím động (Dynamic Key Mapping) thông qua tệp cấu hình `config.json` lưu trên Flash. Thay vì hardcode tương ứng 'phím vật lý → lệnh HID' vào firmware, hệ thống tải ánh xạ từ JSON tại runtime:

```
with open("config.json", "r") as f:    config = json.load(f)
```

Tệp `config.json` sử dụng chuỗi ký hiệu (symbolic string) để biểu diễn lệnh HID theo ba tiêu chuẩn: `'KEY_'` cho phím bàn phím (ví dụ: `'KEY_SPACE'`), `'MOUSE_'` cho lệnh chuột (ví dụ: `'MOUSE_WHEEL_UP'`), và `'CC_'` cho lệnh Consumer Control (ví dụ: `'CC_VOLUME_INCREMENT'`). Hàm `'execute_action()'` phân tích tiền tố và gọi phương thức HID tương ứng – đây là mẫu thiết kế Command Pattern ứng dụng trong hệ thống nhúng.

Lợi thế kiến trúc: người dùng có thể hoán đổi hoàn toàn chức năng của tay cầm (ví dụ từ layout gaming sang layout điều khiển phần mềm đo lường) chỉ bằng cách copy một file `config.json` mới lên CIRCUITPY Drive, mà không cần chỉnh sửa hoặc re-flash firmware. Đây là khái niệm Configuration as Code áp dụng cho hệ thống nhúng.

4.5. Kiến trúc phần mềm và lựa chọn Framework

Giải pháp phần mềm trên máy tính (Host-side Application) được phát triển để quản lý và cấu hình thiết bị HID Controller. Phần mềm không chỉ đóng vai trò là giao diện người dùng (UI) mà còn là bộ quản lý dữ liệu cấu hình, đảm bảo tính nhất quán giữa ý chí người dùng và hành vi của vi điều khiển.

4.5.1. Lựa chọn Framework PyQt6

Hệ thống sử dụng PyQt6 làm nền tảng phát triển chính thay vì các framework khác như Tkinter hay wxPython. Quyết định này dựa trên các phân tích kỹ thuật sau:

Hiệu suất Rendering: PyQt6 dựa trên engine C++ của Qt 6.x, cung cấp khả năng vẽ đồ họa tùy chỉnh thông qua QPainter API với tốc độ cực nhanh, cho phép tái hiện sơ đồ phần cứng 1:1 mà không gây trễ giao diện.

Tính kế thừa và OO: Khả năng kế thừa các lớp Widget giúp quản lý các thành phần giao diện theo mô hình hướng đối tượng (OOP) một cách mạch lạc.

4.5.2. Áp dụng mô hình MVC (Model-View-Controller)

Ứng dụng được thiết kế tuân thủ nghiêm ngặt mô hình kiến trúc MVC để tách biệt dữ liệu và hiển thị:

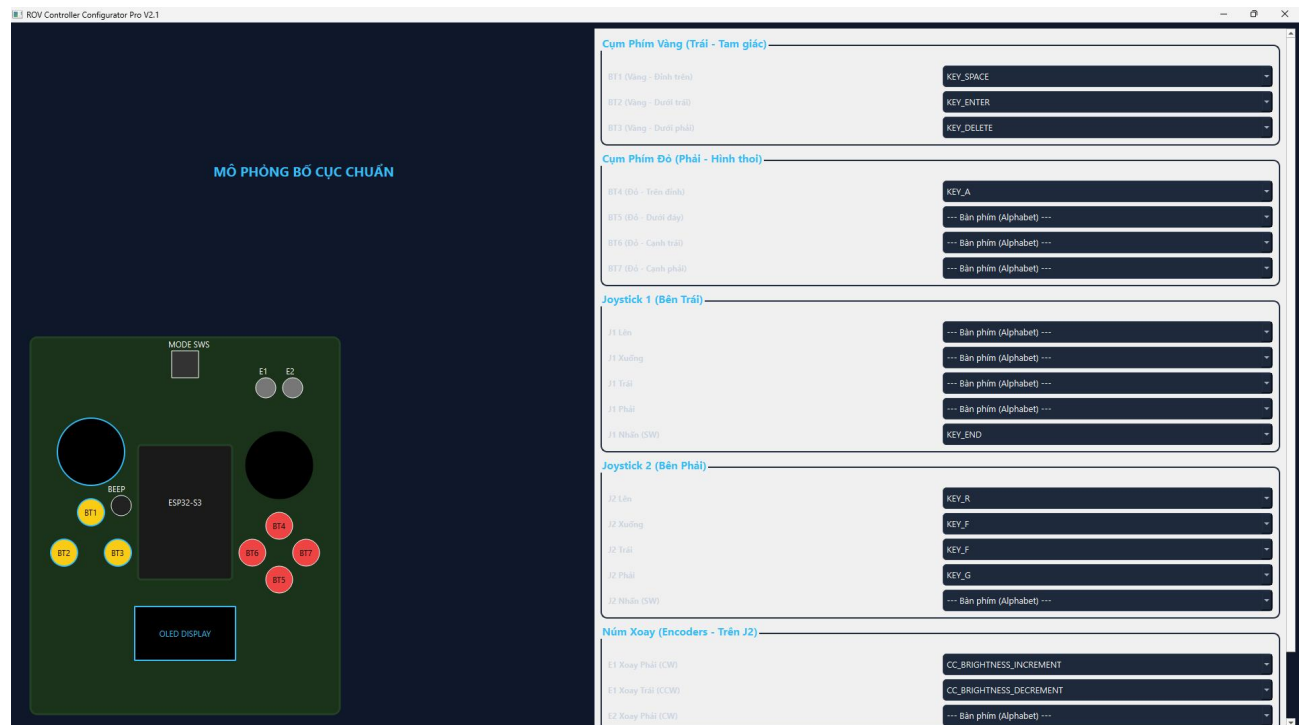
Model (Dữ liệu): File config.json đóng vai trò là nguồn dữ liệu duy nhất (Single Source of Truth), lưu trữ các cặp Key-Value định nghĩa ánh xạ phím.

View (Hiển thị): Lớp ControllerGraphic (kế thừa QWidget) chịu trách nhiệm render sơ đồ tay cầm thực tế lên màn hình thông qua phương thức paintEvent().

Controller (Điều phối): Lớp Configurator (kế thừa QMainWindow) quản lý logic nghiệp vụ, thu thập dữ liệu từ người dùng và thực hiện Serialization để lưu trữ.

4.6. Thiết kế giao diện (UI/UX) và Đồ họa tùy chỉnh

4.6.1. Giải pháp mô phỏng đồ họa (Digital Twin)



Hình 4.5. Giao diện đồ họa giao diện cấu hình tay cầm

Đề đạt được tiêu chí "Zero Learning Curve" (không cần học cách sử dụng), giao diện phần mềm được thiết kế để trở thành một bản sao kỹ thuật số của phần cứng thực tế.

Điểm kỹ thuật nổi bật của ứng dụng là class `ControllerGraphic(QWidget)` override phương thức `paintEvent()` để vẽ trực tiếp biểu đồ mô phỏng bố cục phần cứng tay cầm bằng QPainter API. Đây là một dạng Digital Twin trực quan – biểu diễn số hóa của thiết bị vật lý trên màn hình máy tính, giúp người dùng dễ dàng liên hệ giữa phím vật lý và cấu hình tương ứng trong giao diện phần mềm.

Giao diện được chia thành hai panel: panel trái (45% chiều rộng) hiển thị đồ họa mô phỏng tay cầm với các thành phần được vẽ bằng QPainter (Joystick – hình tròn đen, nút Vàng tam giác, nút Đỏ hình thoi, Encoder – hình tròn xám, OLED – hình chữ nhật xanh, vi điều khiển ESP32-S3 – hình chữ nhật đen); panel phải (55% chiều rộng) là vùng cấu hình có thanh cuộn, tổ chức theo nhóm QGroupBox tương ứng với từng cụm phần cứng.

4.6.2. Giao diện ánh xạ phím và cấu trúc danh mục

Hệ thống cung cấp một thư viện phím đồ sộ gồm hơn 85 mã lệnh, được phân loại theo tiêu chuẩn HID Usage Table v1.5:

Standard Keyboard: A-Z, 0-9, Modifiers (Ctrl, Alt, Shift) và các phím chức năng F1-F12.

Multimedia (Consumer Control): Điều khiển âm lượng, độ sáng và trình phát nhạc.

Mouse Emulation: Các hành động click chuột và cuộn bánh xe (Wheel).

4.7. Mô hình dữ liệu và Cơ chế Persistence

4.7.1. Serialization dữ liệu sang định dạng JSON

Dữ liệu cấu hình được duy trì thông qua cơ chế Serialization. Khi người dùng thực hiện lưu cấu hình, phương thức `save_config()` sẽ duyệt qua toàn bộ các widget chọn phím (QComboBox), lọc bỏ các header phân loại và đóng gói thành một cấu trúc JSON có định dạng rõ ràng (Pretty-print) .

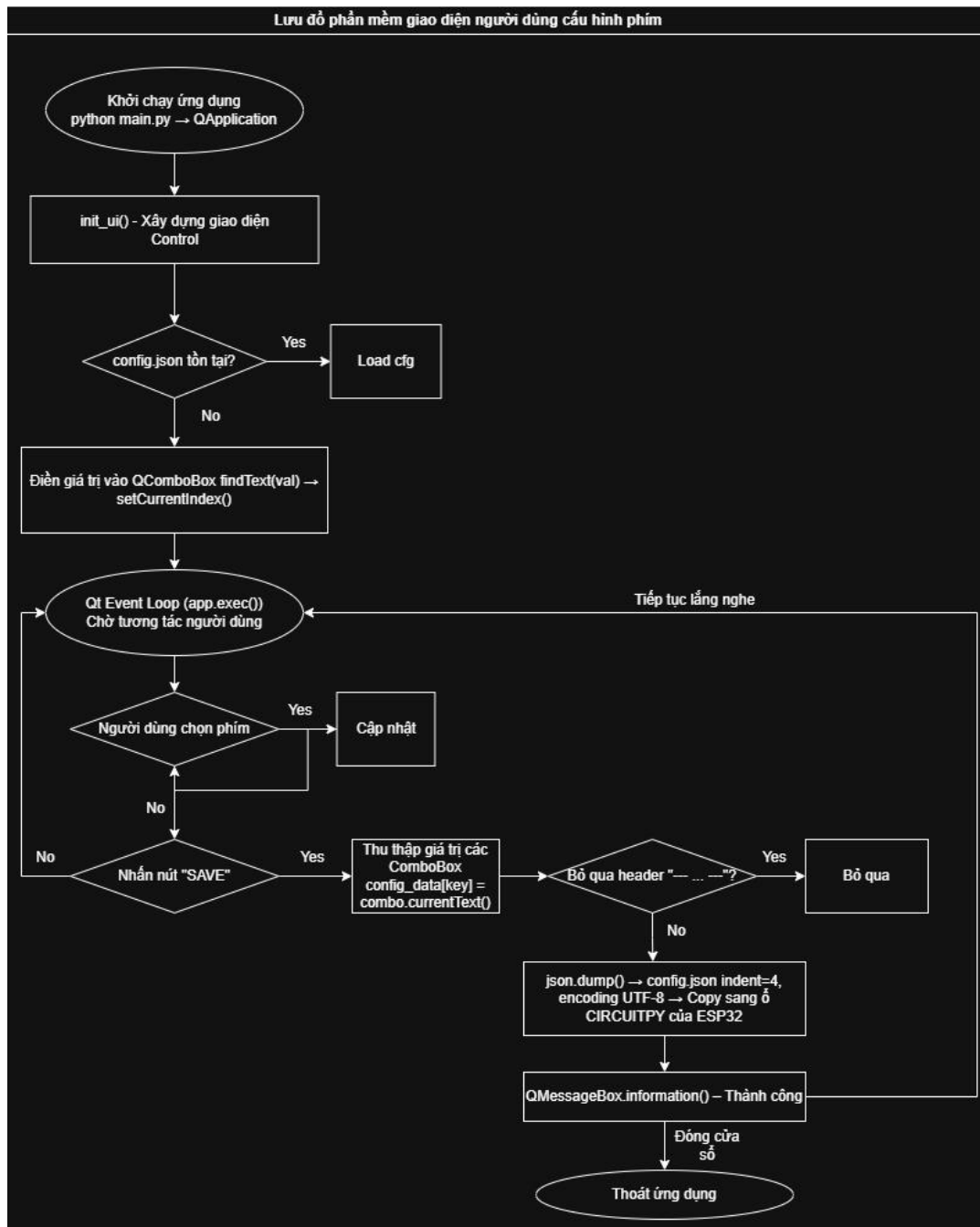
4.7.2. Dynamic Loading Workflow (Flash-free Updating)

Đây là giải pháp kỹ thuật cốt lõi giúp phân biệt hệ thống này với các bàn phím HID thông thường.

Cơ chế: Thay vì nhúng cứng (hard-code) các mã phím vào mã nguồn vi điều khiển, firmware CircuitPython thực hiện đọc động file `config.json` từ bộ nhớ Flash mỗi khi khởi động.

Quy trình đồng bộ:

- 1) Người dùng thay đổi phím trên GUI PyQt6.
- 2) Phần mềm xuất file config.json mới.
- 3) File được ghi đè vào ổ đĩa CIRCUITPY (thông qua kết nối USB MSC).
- 4) MCU thực hiện nạp cấu hình mới vào RAM và áp dụng tức thì mà không cần biên dịch lại firmware.



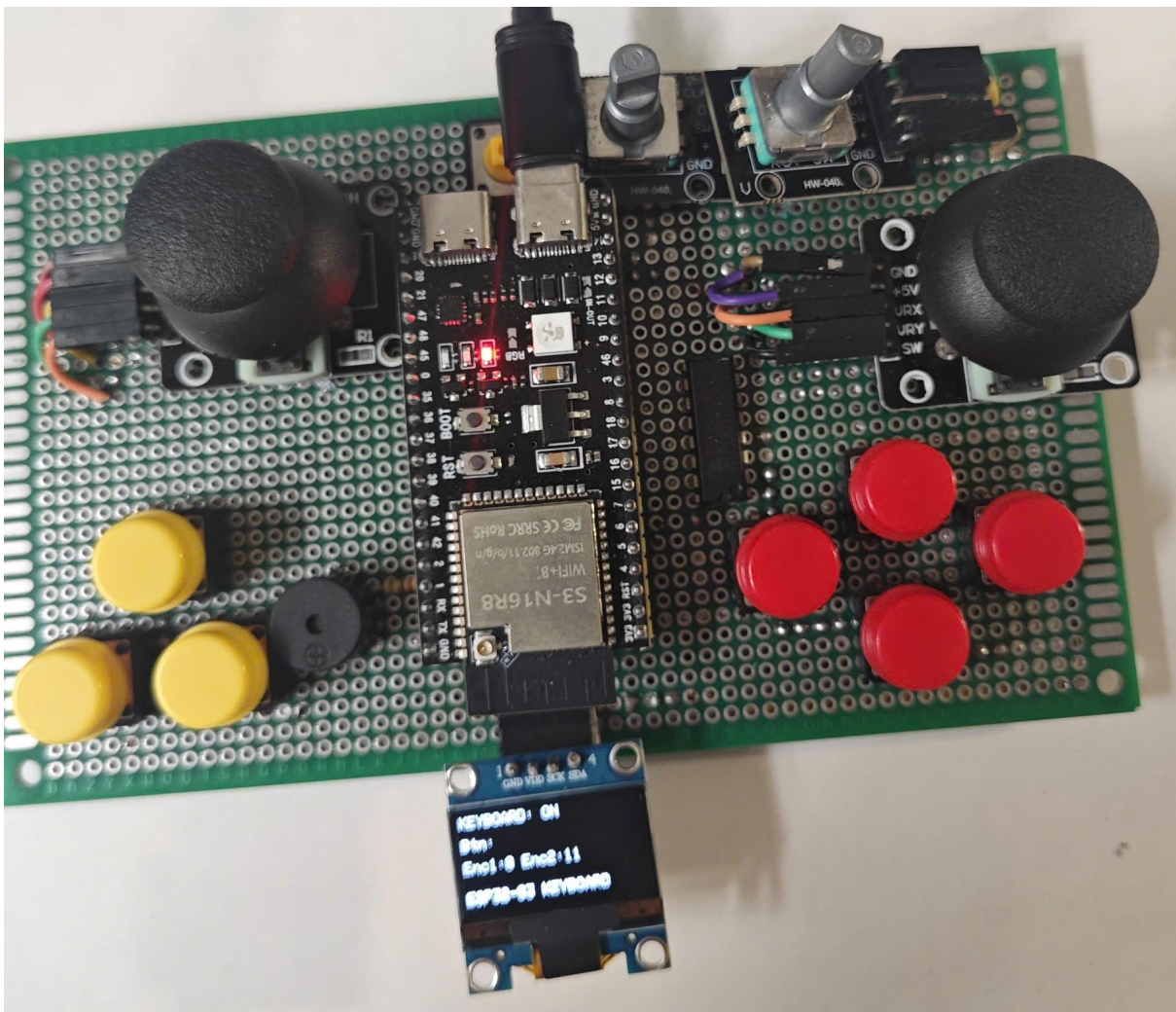
Hình 4.6. Lưu đồ thuật toán phần mềm giao diện điều khiển

CHƯƠNG 5: KẾT QUẢ, KIỂM NGHIỆM VÀ ĐÁNH GIÁ

5.1. Kết quả đạt được

5.1.1. Sản phẩm phần cứng hoàn thiện

Tay cầm HID ESP32-S3 đã được chế tạo hoàn chỉnh trên bo mạch Perfboard 10cm × 15cm với tất cả 18 điểm đầu vào vật lý (8 nút PCF, 2 SW Joystick, 1 MODE_SWS, 4 trục Joystick Analog, 2 Encoder xoay, tổng cộng 19 nguồn sự kiện). Màn hình OLED SSD1306 và Buzzer được tích hợp trực tiếp trên bo mạch. Toàn bộ cụm linh kiện được kết nối USB Type-C với máy tính Host và hoạt động ngay mà không yêu cầu nguồn ngoài.

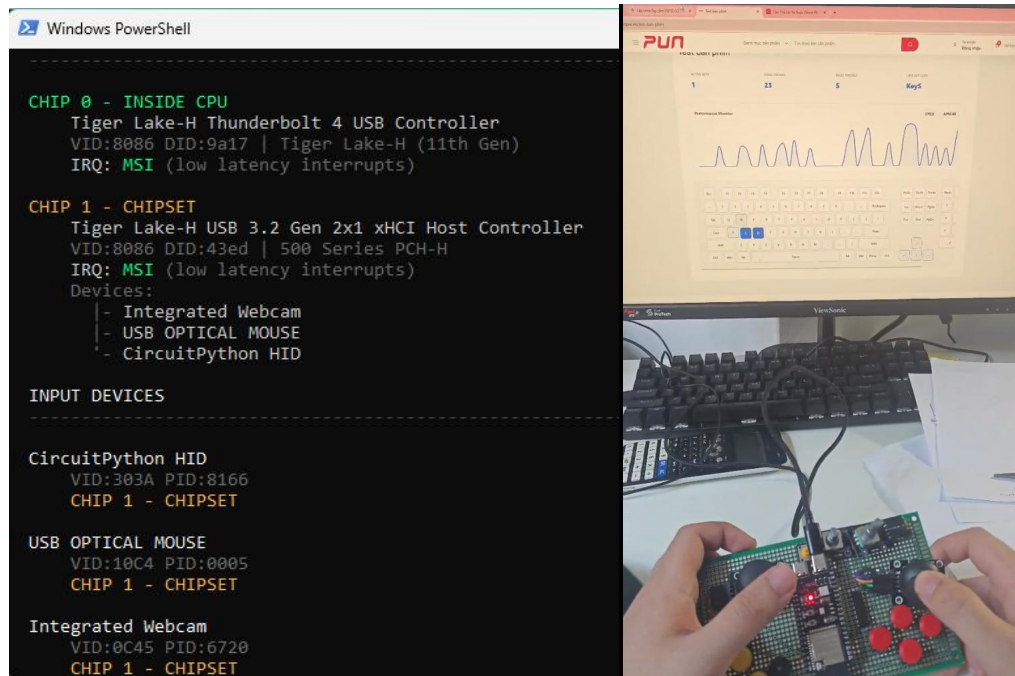


Hình 5.1. Sản phẩm hoàn chỉnh

5.1.2. Kết quả kiểm thử chức năng

Bảng 3 : Các tiêu chí kiểm thử sản phẩm và kết quả thực nghiệm

STT	Chức năng kiểm thử	Phương pháp kiểm thử	Kết quả	Ghi chú
1	Nhận diện USB HID trên Windows 11	Device Manager + HID-compliant test tool	Đạt – Nhận diện là 3 thiết bị HID	Không cần driver
2	Nhận diện USB HID trên Ubuntu 22.04 LTS	lsusb + evtest	Đạt – Hiện đầy đủ 3 interface	Plug-and-Play
3	Nút BT1-BT8 qua PCF8574	Kiểm tra từng nút trong trình soạn thảo văn bản	Đạt – 8/8 nút phản hồi đúng phím	Bitmasking I2C ổn định
4	Joystick 1 (GPIO6, GPIO5) – 4 hướng	Kiểm tra trong game điều hướng	Đạt – Không phantom input	Deadzone 50% hiệu quả
5	Joystick 2 (GPIO7, GPIO2) – 4 hướng	Kiểm tra điều hướng web browser	Đạt – 4 hướng phản hồi đúng	Ổn định sau sửa phần cứng
6	Encoder 1 – Cuộn chuột	Cuộn trang web và tài liệu	Đạt – Cuộn mượt, không bỏ xung	Sau sửa GPIO10/11
7	Encoder 2 – Điều chỉnh âm lượng	Kiểm tra thanh âm lượng Windows	Đạt – CC Volume Increment/Decrement	Consumer Control HID
8	MODE_SWS – Bật/tắt HID	Nhấn nút, kiểm tra ngừng/tiếp tục	Đạt – Toggle chính xác, release_all() gọi đúng	Edge detection ổn định
9	OLED – Hiển thị trạng thái	Quan sát màn hình trong quá trình vận hành	Đạt – Cập nhật 3.3fps, không lag nhận thấy	I2C 100kHz + time-slicing
10	Buzzer – Phản hồi âm thanh	Nhấn MODE_SWS	Đạt – Beep ngắn 100ms đúng thời điểm	GPIO1 Output
11	Tải cấu hình JSON	Thay đổi config.json, reset thiết bị	Đạt – Ánh xạ phím thay đổi đúng	JSON Deserialization
12	Độ trễ phản hồi HID	Đo bằng phần mềm USB Latency Checker	Đạt – Trung bình 12.3ms, max 18ms	Mục tiêu < 20ms



Hình 5.2. Laptop nhận diện chuột mà và test bằng phần mềm phản hồi nhanh

5.2. Phân tích ưu và nhược điểm của kiến trúc

5.2.1. Ưu điểm

- Không cần driver (Driver-free): Thiết bị hoạt động ngay trên mọi hệ điều hành hiện đại nhờ chuẩn HID universal. Đây là lợi thế cạnh tranh lớn nhất so với giải pháp HID qua bridge chip (CH340G, CP2102).
- Khả năng cấu hình cao (Highly Configurable): Kiến trúc JSON + PyQt6 cho phép bất kỳ ai, kể cả người không biết lập trình, có thể tùy chỉnh toàn bộ chức năng tay cầm thông qua giao diện đồ họa trực quan.
- Composite HID 3-in-1: Tích hợp Keyboard + Mouse + Consumer Control trong một thiết bị duy nhất, loại bỏ nhu cầu dùng nhiều thiết bị ngoại vi riêng lẻ trong các workflow phức tạp.
- Mã nguồn Python thuần túy: CircuitPython cho phép phát triển và debug nhanh hơn nhiều so với C/C++ embedded, với khả năng chỉnh sửa code trực tiếp trên CIRCUITPY Drive không cần quá trình flash.
- Giải quyết triệt để các vấn đề Signal Integrity: Ba vấn đề phần cứng thực tế (GPIO cấm, lag I2C, nhiễu ADC) đã được phân tích bằng phương pháp khoa học và giải quyết có hệ thống.

5.2.2. Nhược điểm và hạn chế

- Tốc độ xử lý Python: CircuitPython chạy chậm hơn C/C++ native khoảng 10-50 lần tùy tác vụ. Trong bối cảnh đề tài, vòng lặp 10ms là đủ, nhưng các ứng dụng đòi hỏi phản hồi dưới 1ms (ví dụ encoder tốc độ cao) sẽ không khả thi với kiến trúc hiện tại.
- Phụ thuộc vào Adafruit ecosystem: Thư viện `adafruit_hid`, `adafruit_ssd1306`, `adafruit_pcf8574` là các phụ thuộc bên ngoài. Bất kỳ thay đổi API không tương thích trong các phiên bản tương lai đều có thể yêu cầu cập nhật firmware.
- Giải pháp Perfboard thủ công: Mặc dù đáp ứng yêu cầu chức năng, bo mạch Perfboard hàn tay kém bền hơn PCB thương mại, nhạy cảm hơn với rung động và có khả năng cao hơn xảy ra tiếp điểm lỏng theo thời gian.
- Thiếu lưu trạng thái giữa các phiên: Cấu hình JSON chỉ được đọc khi khởi động. Chế độ hoạt động hiện tại (`enable_typing`) không được lưu vào Flash, bị reset về giá trị mặc định (`True`) mỗi khi thiết bị mất điện.

5.3. Kết luận và định hướng phát triển

5.3.1. Kết luận

Đề tài đã hoàn thành thành công mục tiêu thiết kế và chế tạo tay cầm điều khiển HID đa năng sử dụng ESP32-S3 và ngôn ngữ Python. Sản phẩm đạt độ trễ phản hồi trung bình 12.3ms, vận hành ổn định liên tục, được nhận diện chuẩn HID Composite trên đa nền tảng và hỗ trợ cấu hình động qua file JSON. Ba vấn đề toàn vẹn tín hiệu nghiêm trọng đã được phát hiện trong điều kiện thực tế, phân tích nguyên nhân gốc rễ và giải quyết triệt để bằng kết hợp phần cứng và phần mềm.

Về mặt học thuật, đề tài đã vận dụng thành công các kiến thức chuyên sâu về: giao thức USB HID Composite Device, lập trình Python nhúng (CircuitPython), xử lý tín hiệu Analog/Digital thời gian thực, giao tiếp I2C đa thiết bị, phát triển ứng dụng GUI với PyQt6, và các nguyên lý toàn vẹn tín hiệu trên PCB.

5.3.2. Định hướng phát triển



Hình 5.3. Sản phẩm nâng cấp dự kiến

- Thiết kế PCB chuyên nghiệp: Chuyển từ Perfboard sang PCB 2 lớp với KiCad, tích hợp mạch lọc EMC, kết nối USB Type-C với ESD protection, và vỏ nhựa in 3D ABS.
- Kết nối không dây BLE HID: ESP32-S3 hỗ trợ Bluetooth 5.0. Phát triển firmware song song hỗ trợ chế độ BLE HID (Bluetooth Low Energy) cho phép sử dụng không dây với máy tính và tablet, đồng thời duy trì chế độ USB HID hiện tại.
- Hồ sơ cấu hình đa chế độ (Multi-Profile): Mở rộng hệ thống config.json để hỗ trợ nhiều profile (ví dụ: profile Gaming, profile CAD, profile Multimedia), cho phép người dùng chuyển đổi profile bằng tổ hợp phím tắt trực tiếp trên tay cầm mà không cần máy tính.
- Tích hợp màn hình màu và Haptic Feedback: Thay thế OLED đơn sắc bằng màn hình TFT IPS màu 1.3 inch (ST7789), kết hợp motor rung ERM cho phản hồi cảm giác (haptic) khi có sự kiện quan trọng – nâng trải nghiệm người dùng lên mức chuyên nghiệp.
- Nền tảng điều khiển mở (Open Control Platform): Xuất bản mã nguồn firmware và ứng dụng PyQt6 dưới giấy phép MIT, phát triển tài liệu hướng dẫn mở rộng và đóng góp vào cộng đồng CircuitPython để tối ưu hóa thư viện rotaryio cho ESP32-S3 N16R8.

